MICROSOFT CERTIFIED: FABRIC DATA ENGINEER ASSOCIATE

Exam DP-700 Study Guide

Implementing Data Engineering Solutions Using Microsoft Fabric

EXAM **DP-700**	SKILLS OUTLINE VERSION **19 October 2026**
TIME TO COMPLETE **100 minutes, proctored**	PASSING SCORE **700 of 1000**
LEVEL **Intermediate · Associate**	RENEWAL **Free online, every 12 months**

OneLake · Spark & PySpark · Pipelines · Eventstream · Eventhouse & KQL · Mirroring · Delta Lake · CI/CD · Monitoring · Performance tuning

Every Microsoft link in this guide carries the Microsoft Learn Student Ambassadors Contributor ID `wt.mc_id=studentamb_657938`. Language-locale segments have been removed and the ID is appended with `?` or `&` exactly as the Registered Member Handbook requires, so you can copy any link straight out of this document and share it.

Built from the official skills outline published on the Microsoft Learn DP-700 study guide page.

Table of contents

Chapter 0 — How to use this guide

0.1 What DP-700 actually certifies

0.2 The three skill areas and their weights

0.3 Exam logistics you should know before booking

0.4 A six-week study plan

0.5 How to read the callouts

Chapter 1 — The Fabric platform for data engineers

1.1 What you are building on

1.2 Tenant, capacity, workspace, item

1.3 The medallion architecture

1.4 The data stores

1.5 Delta Lake — the format underneath everything

Chapter 2 — Configure Microsoft Fabric workspace settings

2.1 Spark workspace settings

2.2 Domain workspace settings

2.3 OneLake workspace settings

2.4 Apache Airflow job settings

2.5 Chapter 2 self-check

2.6 Learn more

Chapter 3 — Implement lifecycle management in Fabric

3.1 Version control

3.2 Database projects

3.3 Deployment pipelines

3.4 Chapter 3 self-check

3.5 Learn more

Chapter 4 — Configure security and governance

4.1 The mental model

4.2 Workspace-level access

4.3 Item-level access

4.4 OneLake security: folder and file level

4.5 Row-level, column-level, and object-level security

4.6 Dynamic data masking

4.7 Sensitivity labels and endorsement

4.8 Audit logs

4.9 Chapter 4 self-check

4.10 Learn more

Chapter 5 — Orchestrate processes

5.1 Choosing the tool

5.2 Orchestration patterns

5.3 Parameters and dynamic expressions
5.4 Schedules and event-based triggers
5.5 Chapter 5 self-check
5.6 Learn more

Chapter 6 — Design and implement loading patterns

6.1 Full versus incremental
6.2 The watermark pattern
6.3 Preparing data for a dimensional model
6.4 Loading patterns for streaming data
6.5 Chapter 6 self-check
6.6 Learn more

Chapter 7 — Ingest and transform batch data

7.1 Choosing the transformation engine
7.2 OneLake shortcuts
7.3 Mirroring
7.4 Ingesting with pipelines
7.5 Transforming: the same job in three languages
7.6 Duplicate, missing, and late-arriving data
7.7 Chapter 7 self-check
7.8 Learn more

Chapter 8 — Ingest and transform streaming data

8.1 The Real-Time Intelligence architecture
8.2 Choosing a streaming engine
8.3 Processing with Eventstream
8.4 Native tables versus OneLake shortcuts in Real-Time Intelligence
8.5 Processing with KQL inside the eventhouse
8.6 Windowing functions
8.7 Chapter 8 self-check
8.8 Learn more

Chapter 9 — Monitor Fabric items

9.1 Where to look
9.2 Monitoring ingestion
9.3 Monitoring transformation
9.4 Monitoring semantic model refresh
9.5 Configuring alerts
9.6 Chapter 9 self-check
9.7 Learn more

Chapter 10 — Identify and resolve errors

10.1 Pipeline errors
10.2 Dataflow Gen2 errors
10.3 Notebook and Spark errors
10.4 Eventhouse and Eventstream errors
10.5 T-SQL errors

10.6 Shortcut errors
10.7 Chapter 10 self-check

Chapter 11 — Optimize performance

11.1 Optimise a lakehouse table
11.2 Optimise Spark
11.3 Optimise a pipeline
11.4 Optimise a data warehouse
11.5 Optimise Eventstream and Eventhouse
11.6 Optimise query performance
11.7 Chapter 11 self-check
11.8 Learn more

Chapter 12 — The hands-on lab track

Chapter 13 — Practice questions

Chapter 14 — Exam-day checklist and quick reference

14.1 The week before
14.2 The 60-second memory dump
14.3 Reading exam questions
14.4 Glossary
14.5 Where to go next

Appendix A — One-page decision tables

A.1 Which ingestion mechanism?
A.2 Which transformation engine?
A.3 Which trigger?
A.4 Which fix for slowness?
A.5 Which security control?

Chapter 0 — How to use this guide

0.1 What DP-700 actually certifies

Passing **Exam DP-700: Implementing Data Engineering Solutions Using Microsoft Fabric** earns the Microsoft Certified: Fabric Data Engineer Associate credential. Microsoft's description of the role:

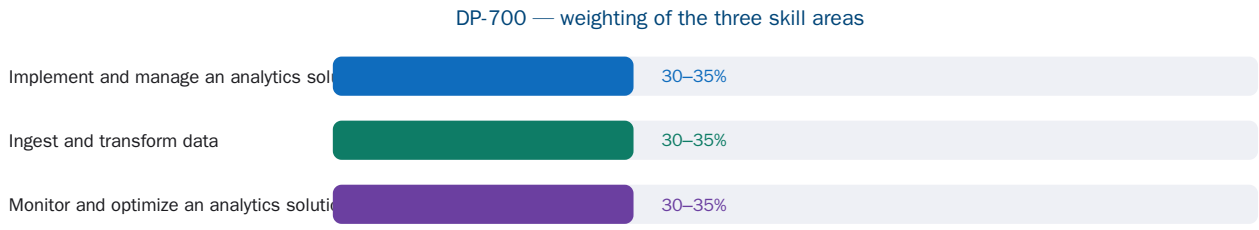
> As a candidate for this exam, you should have subject matter expertise with data loading patterns, data architectures, and orchestration processes.

Your three responsibilities, from the official DP-700 study guide:

- Ingesting and transforming data
- Securing and managing an analytics solution
- Monitoring and optimizing an analytics solution

You are expected to manipulate and transform data with **SQL**, **PySpark**, and **KQL**. Where DP-600 asks "is this model correct and fast?", DP-700 asks "does this pipeline run, recover, scale, and cost what it should?"

0.2 The three skill areas and their weights



Skills measured as of 19 October 2026. Unlike DP-600, the three areas are almost evenly weighted — you cannot skip one.

Skill area	Weight	Chapters in this guide
Implement and manage an analytics solution	30–35%	2, 3, 4, 5
Ingest and transform data	30–35%	6, 7, 8
Monitor and optimize an analytics solution	30–35%	9, 10, 11

KEY TAKEAWAY

Every area is worth roughly a third. A candidate who is excellent at pipelines but has never opened the Monitoring hub or tuned a Spark job will fail. Give monitoring, troubleshooting, and optimisation the same study time as ingestion.

0.3 Exam logistics you should know before booking

Item	Detail
Duration	100 minutes to complete the assessment; the appointment is longer — see exam duration and experience
Passing score	700 on a scale of 1000, scaled — see exam scoring and score reports
Question count	Not published. Expect roughly 40–60 items, usually including a case study
Question types	Multiple choice, multi-select, drag-and-drop, hot area, build-list, and possible interactive components
Languages	English, Japanese, Chinese (Simplified), Chinese (Traditional), German, French, Italian, Korean, Spanish, Portuguese (Brazil)
Delivery	Proctored, online or at a Pearson VUE test centre
Retake	24 hours after a first failure, longer for later attempts — retake policy
Renewal	Free online assessment every 12 months — renew your certification
Accommodations	Request accommodations for extra time or assistive technology

> **TIP**
>
> Take the free DP-700 practice assessment early to find your weak area, and again in the final week. Use the exam sandbox from the DP-700 exam page so the interface is familiar.

0.4 A six-week study plan



Roughly 6–8 hours per week. The labs are not optional — DP-700 questions are written from operational experience.

Week	Focus	This guide	Microsoft Learn
1	Platform, OneLake, lakehouse, Spark basics	Ch. 1, 2	Get started with Microsoft Fabric · Implement a Lakehouse
2	Batch ingestion and transformation	Ch. 6, 7	Ingest data with Microsoft Fabric
3	Streaming and Real-Time Intelligence	Ch. 8	Implement Real-Time Intelligence
4	Security, governance, CI/CD, orchestration	Ch. 3, 4, 5	Manage a Fabric environment · Secure and govern analytics data
5	Monitoring, troubleshooting, optimisation	Ch. 9, 10, 11	Monitor activities in Fabric
6	Practice, weak spots, exam readiness	Ch. 12, 13, 14	Practice assessment · Exam Readiness Zone

0.5 How to read the callouts

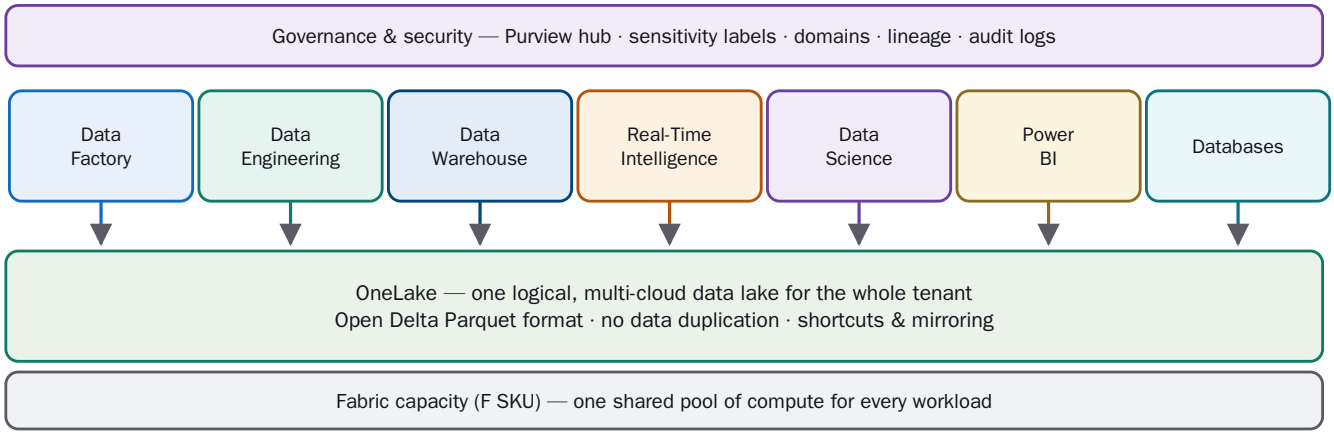
Callout	What it means
Exam focus	Wording from the official skills outline
Common exam trap	A distinction the exam deliberately tests
Know the difference	Two features that look alike under time pressure
Key takeaway	The sentence to remember
Hands-on lab	Do it in a real workspace
Tip / **Watch out**	Practical advice and gotchas

> **HANDS-ON LAB**
>
> Start a Microsoft Fabric trial and create one workspace called `dp700-lab` with a lakehouse, a warehouse, and an eventhouse. You will use all three throughout this guide.

Chapter 1 — The Fabric platform for data engineers

1.1 What you are building on

Microsoft Fabric is a SaaS analytics platform: one capacity of compute, one lake of storage, many workloads.



One capacity, one lake. As a data engineer you live mostly in Data Factory, Data Engineering, and Real-Time Intelligence — but everything you write lands in OneLake.

The three facts that shape every design decision:

- **Everything persists to OneLake in open Delta Parquet.** A table written by a Spark notebook is instantly readable by T-SQL, by KQL through shortcuts, and by a Direct Lake semantic model.
- **Compute is shared.** Your Spark job and someone else's report draw Capacity Units from the same F SKU, and sustained overuse causes throttling.
- **Governance is platform-wide.** Sensitivity labels, lineage, endorsement, domains, and audit logs cover every item type.

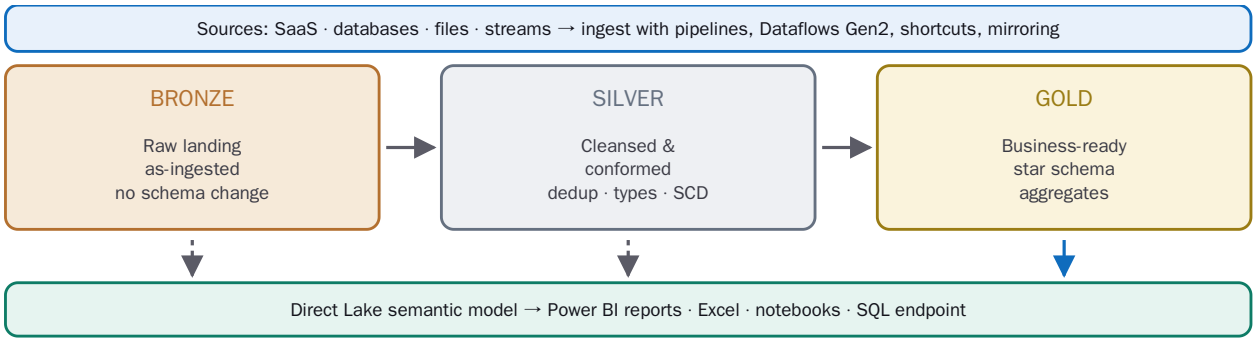
1.2 Tenant, capacity, workspace, item

Level	What it controls	Where you configure it
Tenant	Global feature switches, who can create what	Admin portal
Capacity (F2–F2048)	The compute pool, and which workspaces use it	Capacity settings, scale capacity
Domain	Business-area grouping of workspaces	Domains
Workspace	Items, roles, Spark settings, Git binding	Workspaces
Item	The lakehouse, pipeline, notebook, eventhouse itself	Item settings and permissions

Operational facts worth remembering: CU usage is **smoothed** over a rolling window; background operations (pipelines, Spark) are smoothed over 24 hours while interactive operations are smoothed over minutes; a

capacity can be paused and resumed, which stops both billing and all running jobs; the Capacity Metrics app is the only supported way to see what consumed what.

1.3 The medallion architecture

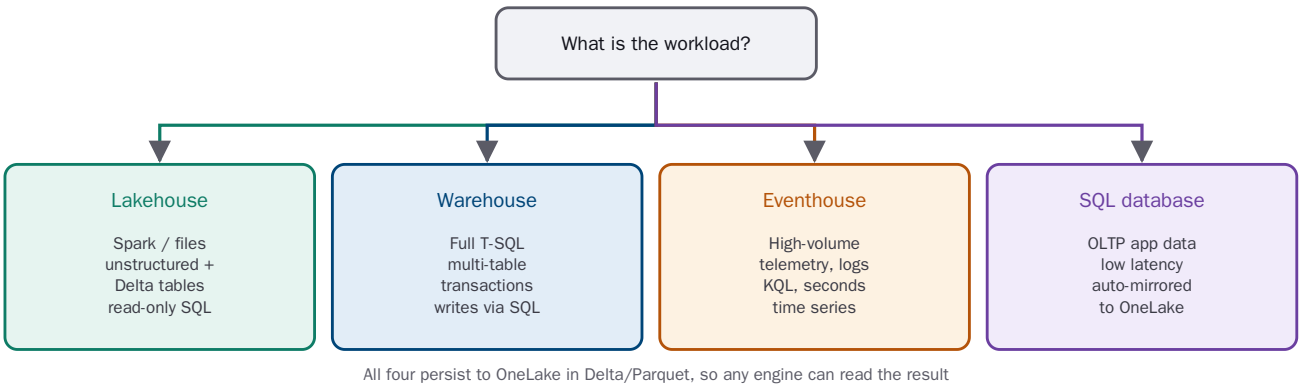


Bronze keeps the raw truth so you can replay; Silver makes it correct; Gold makes it useful.

Medallion is the reference layout used by almost every DP-700 scenario:

Layer	Content	Typical engine	Typical table properties
Bronze	Raw, append-only, as ingested, partitioned by load date	Pipeline Copy, Eventstream, mirroring	No constraints, keep source columns, add `_ingest_ts` and `_source_file`
Silver	Cleansed, typed, deduplicated, conformed, SCD applied	Notebook or Dataflow Gen2	Delta `MERGE`, enforced schema, quality quarantine
Gold	Star schemas, aggregates, serving tables	Notebook or T-SQL	Optimised and V-Ordered, statistics maintained

1.4 The data stores



Four analytical stores. Everything lands in OneLake as Delta, so the question is which engine owns the write path.

	Lakehouse	Warehouse	Eventhouse (KQL)	SQL database
Write with	Spark, pipelines, Dataflows Gen2	T-SQL (`INSERT` / `MERGE` / `COPY INTO`)	Streaming ingestion, `.ingest`	Full OLTP T-SQL
Multi-table transactions	No	Yes	No	Yes
Files as well as tables	Yes	No	No	No
Best at	Engineering at scale, ML, mixed formats	Dimensional serving, SQL teams	Telemetry, logs, time series	Operational applications
SQL endpoint	Read-only	Read/write	—	Read/write

Decision guidance: choose a data store · pipeline vs dataflow vs Spark.

1.5 Delta Lake — the format underneath everything

Every lakehouse table is a Delta table: Parquet data files plus a `_delta_log` transaction log that gives you ACID transactions, time travel, and schema enforcement.

```
# Read a specific historical version (time travel)
df = spark.read.format("delta").option("versionAsOf", 12).table("silver.sales")

# Or as of a timestamp
df = spark.read.format("delta").option("timestampAsOf", "2026-09-01
00:00:00").table("silver.sales")
```

```
-- History and maintenance
DESCRIBE HISTORY silver.sales;
OPTIMIZE silver.sales;                -- compact small files (+ V-Order in Fabric)
VACUUM   silver.sales RETAIN 168 HOURS;  -- 7 days, the default minimum
```

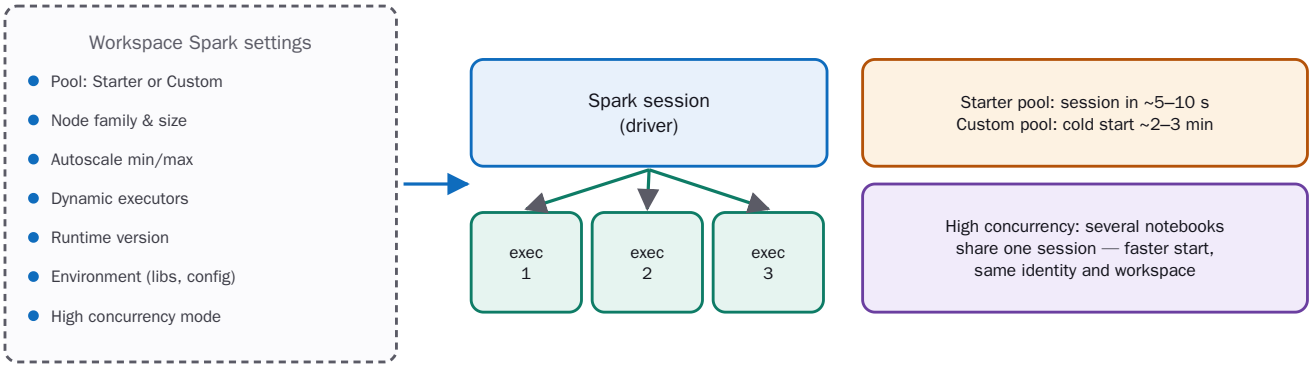
V-Order is a Fabric-specific write optimisation applied on top of Parquet: sorting, encoding, and compression that make reads dramatically faster for Power BI and SQL, at a modest write cost. See Delta optimization and V-Order.

> **KEY TAKEAWAY**
>
> Bronze tables are write-heavy: consider turning V-Order off there. Gold tables are read-heavy: keep V-Order on and run `OPTIMIZE` after loads.

Chapter 2 — Configure Microsoft Fabric workspace settings

> **EXAM FOCUS**
>
> **Configure Microsoft Fabric workspace settings** — configure Spark workspace settings · configure domain workspace settings · configure OneLake workspace settings · configure Apache Airflow workspace settings.

2.1 Spark workspace settings



Workspace Spark settings decide how fast a session starts, how big it can grow, and which libraries are available.

Open **Workspace settings** → **Data Engineering/Science** → **Spark settings**. Reference: Spark workspace administration settings.

Setting	Options	What to know for the exam
Pool	Starter pool or custom pool	Starter pools are pre-warmed: sessions start in seconds. Custom pools cold-start in minutes but let you fix node size and count. See Spark compute and starter pools
Node family / size	Memory-optimised sizes	Bigger nodes for wide shuffles and skew; more nodes for parallel scans
Autoscale	Min and max nodes	The job scales within the range; the max bounds your worst-case CU burn
Dynamic allocation	Min/max executors	Executors are added and released within a session as stages demand
Runtime version	e.g. Runtime 1.3 (Spark 3.5 / Delta 3.x)	Pinned per workspace or per environment — see Fabric Runtime
Environment	Default environment for the workspace	Packages, Spark properties, and resources in one reusable item — environments
High concurrency	On/off for notebooks and pipelines	Multiple notebooks share one session, so they start instantly and use fewer CUs — high concurrency
Automatic log	On/off	Controls automatic logging of Spark events for monitoring

> **KNOW THE DIFFERENCE**
> **Starter pool vs custom pool.** Starter pool = fast start, standard sizing, ideal for interactive development. Custom pool = you control node family, size, and autoscale range, ideal for a heavy production job with predictable shape. "Sessions take three minutes to start" almost always means a custom pool where a starter pool would do.

> **COMMON EXAM TRAP**
> High concurrency shares a **session**, not an identity or a workspace. Notebooks that share a session must be run by the same user against the same workspace, and a crash in one notebook can affect the session. It saves start-up time and CUs; it does not isolate workloads.

Libraries can be attached at three levels — workspace default environment, a specific environment attached to a notebook, or inline with `%pip install` for the session only. Inline installs restart the session. See library management.

2.2 Domain workspace settings

Domains group workspaces by business area (Finance, Supply Chain) so governance can be federated: a domain admin manages endorsement and default settings for their own area without tenant-admin rights.

- A workspace is assigned to **one domain**, optionally within a subdomain.
- Domain admins can set **default sensitivity labels** and manage certification for their domain.

- Domains drive filtering in the OneLake catalog, so users browse "Finance data" rather than 400 workspaces.
- Assignment can be done by workspace, by capacity, or by workspace admin.

2.3 OneLake workspace settings

In **Workspace settings → OneLake** you control how data in this workspace can be reached from outside Fabric — for example whether users may access it through OneLake file explorer, Azure Storage Explorer, or other ADLS-compatible tools. Turning external access off does not change Fabric-internal permissions; it closes the door used by external applications.

Related controls you should be able to place:

Control	Level	Purpose
OneLake access for external apps	Workspace	Allow or block ADLS-compatible clients
OneLake security data access roles	Item (lakehouse)	Folder-, table-, row-, and column-level rules enforced for every engine
Tenant switches for OneLake	Tenant	Org-wide policy, for example external data sharing

2.4 Apache Airflow job settings

Fabric hosts managed **Apache Airflow jobs** for teams that already orchestrate with Airflow DAGs and want to keep that ecosystem while running inside Fabric.

- An Airflow job is a Fabric item; you author DAGs in Python and run them on Fabric-managed Airflow.
- Settings cover the Airflow **version**, the **compute size** of the environment, **extra Python requirements**, **environment variables**, and **Airflow configuration overrides**.
- Plugins and requirements are supplied per job so DAGs stay portable.
- Create one with create an Apache Airflow job.

> **KNOW THE DIFFERENCE**
>
> **Airflow job vs data pipeline.** Use a **data pipeline** for Fabric-native orchestration with a visual designer, built-in activities, and the tightest integration. Use an **Airflow job** when the organisation already has Airflow DAGs, Python-defined dependencies, or cross-platform orchestration that must remain in Airflow. Do not migrate to Airflow "for features" — pipelines cover the exam's orchestration patterns.

2.5 Chapter 2 self-check

1. Notebook sessions take nearly three minutes to start during development. What do you change?
2. Three analysts run small notebooks all day and the capacity is saturated by session overhead. What do you enable?

3. Where do you pin a specific version of `pandas` for every notebook in the workspace?
4. Finance wants to manage endorsement for their own workspaces without tenant-admin rights. What do you configure?
5. Which setting stops Azure Storage Explorer from reading this workspace's OneLake data?

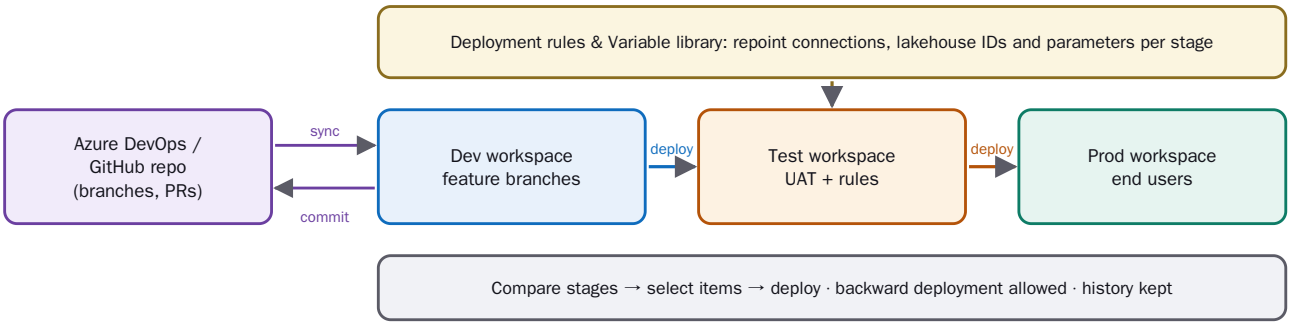
> **Answers.** 1. Use the starter pool instead of a custom pool. 2. High concurrency mode for notebooks. 3. A workspace default environment with that library, or a custom environment attached to the notebooks. 4. A domain with domain admins. 5. The workspace OneLake setting that governs access from external, ADLS-compatible applications.

2.6 Learn more

- Spark workspace administration settings · Spark compute · starter pools
- Environments · library management · Fabric Runtime
- High concurrency mode
- Domains · OneLake security
- Apache Airflow jobs

Chapter 3 — Implement lifecycle management in Fabric

> **EXAM FOCUS**
> **Implement lifecycle management in Fabric** — configure version control · implement database projects · create and configure deployment pipelines.



Git is the source of truth, deployment pipelines promote, and rules or variable libraries re-point each stage at its own data.

3.1 Version control

Git integration binds a workspace to one branch in **Azure DevOps** or **GitHub**. Items serialise to text files, so you get diffs, pull requests, and history.

The workflow: connect the workspace → **Commit** workspace changes to the branch → **Update** to pull branch changes in → resolve conflicts in the Source control pane → use **Branch out to a new workspace** for isolated feature work.

For data engineering items specifically:

- **Notebooks** serialise as `.py` / `.ipynb` with metadata, so code review works properly — see notebook source control and deployment.
- **Pipelines** serialise as JSON definitions.
- **Lakehouses** serialise metadata and shortcuts, **not** the Delta data.
- **Environments** serialise their libraries and Spark configuration.

> **COMMON EXAM TRAP**
> Git carries **definitions, not data**. Updating a test workspace from Git gives you empty lakehouse tables. Populating test data is a separate pipeline or script — a favourite exam distinction.

3.2 Database projects

A **SQL database project** is the declarative, offline representation of a warehouse's schema: tables, views, procedures, functions, and security objects as `.sql` files in a project that can be built and deployed.

- Author in **Visual Studio Code with the SQL Database Projects extension** (or Visual Studio / Azure Data Studio).

- Build produces a **dacpac**; deploy compares the dacpac against the target and generates the change script.
- This gives warehouses the same review-and-promote discipline that `.pbip` gives semantic models.
- It pairs with deployment pipelines: pipelines move the item, database projects manage the schema deltas precisely.

> **KNOW THE DIFFERENCE**
> **Git integration vs database project.** Git integration versions the *Fabric item* and syncs a whole workspace. A database project versions the *schema* of one warehouse and gives you a build-and-compare deployment with a generated migration script. Large SQL teams use both.

3.3 Deployment pipelines

Deployment pipelines promote items between stage workspaces — Development, Test, Production, up to **10** stages.

Concept	Detail
Stage ↔ workspace	One workspace per stage; you must be a workspace Admin to assign it
Compare	Shows New, Different, and Same items before you deploy
Selective deployment	Deploy only the items you choose
Backward deployment	Allowed when the target does not already contain the item
Data	Not deployed. Definitions only
Deployment rules	Per-stage data sources, parameters, and default lakehouse bindings
Variable library	One named variable, a different value per stage, referenced by many items

For automation, drive the same operations from Azure DevOps or GitHub Actions through the Fabric REST APIs. Guidance: CI/CD overview and CI/CD best practices.

> **COMMON EXAM TRAP**
> A notebook deployed to Production still points at the Development lakehouse unless you re-bind it. Use a **default lakehouse deployment rule** or a **variable library** — this is the single most commonly tested lifecycle detail in DP-700.

3.4 Chapter 3 self-check

1. Twelve notebooks must each read the lakehouse of their own stage. What is the least-effort configuration?
2. What is the artefact produced by building a SQL database project?
3. After updating a Test workspace from Git, the tables are empty. Why?

4. Which Fabric feature lets two engineers review each other's pipeline JSON before it reaches Test?
5. Can you deploy from Production back to Test?

> **Answers.** 1. A variable library (or a default lakehouse deployment rule) rather than editing twelve notebooks. 2. A dacpac, deployed by comparing it with the target. 3. Git versions item definitions, not the data in Delta tables. 4. Git integration with pull requests on the branch the workspace is bound to. 5. Yes — backward deployment is supported when the target stage does not already contain the item.

3.5 Learn more

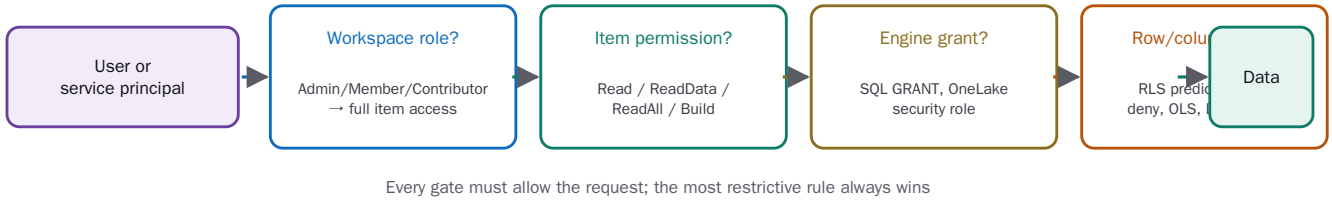
- Git integration · deployment pipelines · get started
- Source control for Fabric Data Warehouse
- Notebook source control and deployment
- Variable libraries · CI/CD best practices
- Training: Implement CI/CD in Microsoft Fabric

Chapter 4 — Configure security and governance

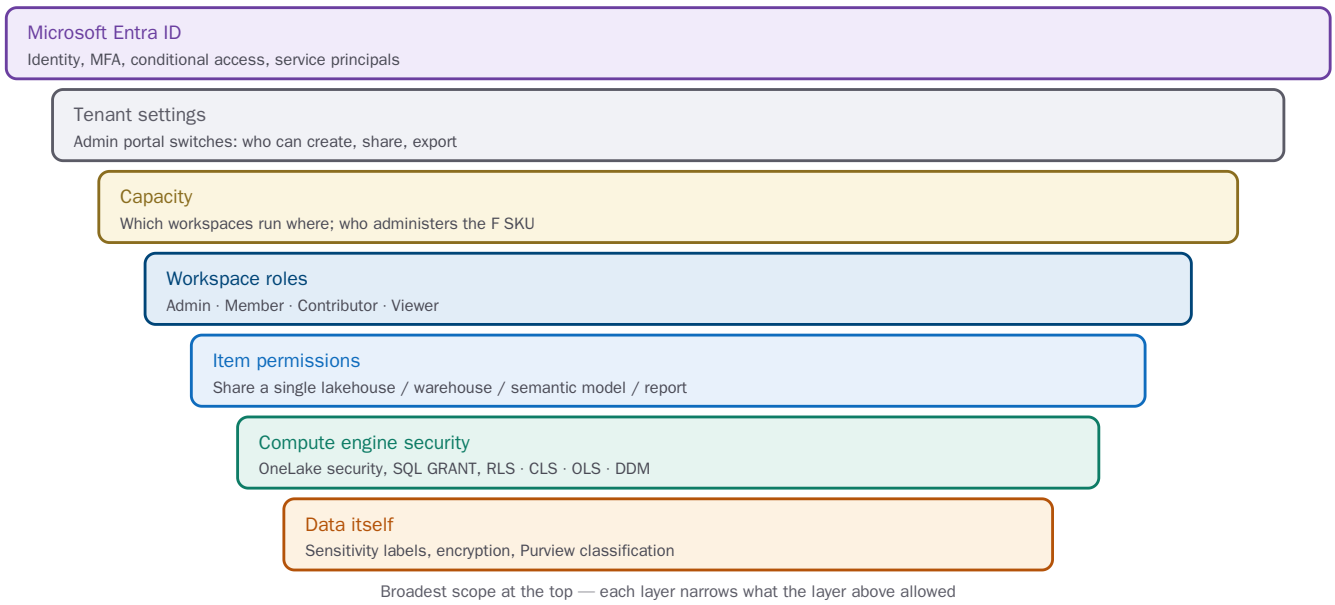
EXAM FOCUS

Configure security and governance — implement workspace-level access controls · implement item-level access controls · implement row-level, column-level, object-level, and folder/file-level access controls · implement dynamic data masking · apply sensitivity labels to items · endorse items · implement and use Microsoft Fabric audit logs · configure and implement OneLake security.

4.1 The mental model



Four gates. Every one must allow the request, and the most restrictive rule wins.



From identity to the data itself — know which control lives at which layer.

4.2 Workspace-level access

Workspace roles grant blanket access to every item in the workspace. Always assign them to Microsoft Entra **security groups**.

Capability	Admin	Member	Contributor	Viewer
Delete workspace, change capacity, manage all roles	✔			
Add Contributors and Viewers, share items, publish apps	✔	✔		
Create/edit/delete items, run pipelines and notebooks	✔	✔	✔	
View items, query through the SQL analytics endpoint	✔	✔	✔	✔
Read raw files with Spark or OneLake APIs	✔	✔	✔	

COMMON EXAM TRAP

A **Viewer** cannot read files with Spark, and cannot run a notebook. If a scenario says "the support team must be able to re-run the failed pipeline", Viewer is wrong — that needs Contributor, or item-level Execute permission.

4.3 Item-level access

Permission	Meaning
Read	See the item and its metadata
ReadData	Query data through the SQL analytics endpoint
ReadAll	Read the underlying OneLake files (Spark, OneLake APIs)
Write	Modify the item definition
Execute	Run the item, for example a notebook or pipeline
Build	Create new content on a semantic model
Reshare	Grant the permission onward

Reference: Fabric permission model.

4.4 OneLake security: folder and file level

OneLake security is the layer that enforces access **for every engine** — Spark, T-SQL, KQL, Direct Lake, and the OneLake APIs.

- You create **data access roles** on a lakehouse and list the folders and tables each role can read.
- Roles are assigned to Entra users and groups.
- Row- and column-level restrictions defined here follow the data into downstream engines.

Read: get started with OneLake security · data access control model · get started with data access roles.

> **KNOW THE DIFFERENCE**
>
> **OneLake security vs SQL `GRANT`.** OneLake security is enforced whatever the tool. `GRANT` / `DENY` on the SQL analytics endpoint binds only callers who come through that endpoint — Spark with `ReadAll` walks straight past it. "Regardless of how they connect" ⇒ OneLake security.

4.5 Row-level, column-level, and object-level security

RLS in a warehouse or SQL endpoint (T-SQL):

```
CREATE FUNCTION Security.fn_RegionPredicate(@Region AS varchar(50))
    RETURNS TABLE WITH SCHEMABINDING
AS
    RETURN SELECT 1 AS granted
           WHERE @Region = (SELECT Region FROM dbo.UserRegion WHERE UserName = USER_NAME())
              OR IS_ROLEMEMBER('DataStewards') = 1;
GO
CREATE SECURITY POLICY Security.RegionFilter
    ADD FILTER PREDICATE Security.fn_RegionPredicate(Region) ON dbo.FactSales
    WITH (STATE = ON);
```

CLS:

```
GRANT SELECT ON dbo.DimEmployee(EmployeeKey, FullName, Department) TO [Analysts];
DENY  SELECT ON dbo.DimEmployee(Salary) TO [Analysts];
```

RLS in an eventhouse (KQL): a row level security policy rewrites every query for the principal.

```
.alter table Telemetry policy row_level_security enable
"Telemetry | where Tenant == current_principal_details()['UserPrincipalName']"
```

OLS hides an entire table or column in a *semantic model*, configured through external tooling over the XMLA endpoint.

References: row-level security · column-level security · SQL granular permissions.

4.6 Dynamic data masking

Dynamic data masking obscures values **at query time** for users who lack the `UNMASK` permission. The stored data is unchanged.

```
ALTER TABLE dbo.DimCustomer
    ALTER COLUMN Email     ADD MASKED WITH (FUNCTION = 'email()');
ALTER TABLE dbo.DimCustomer
    ALTER COLUMN NationalId ADD MASKED WITH (FUNCTION = 'partial(0,"XXX-XX-",4)');
ALTER TABLE dbo.DimCustomer
    ALTER COLUMN Salary    ADD MASKED WITH (FUNCTION = 'random(1000, 9000)');

GRANT UNMASK ON dbo.DimCustomer TO [HRTeam];
```

Mask functions: `default()`, `email()`, `random(a,b)`, `partial(prefix, padding, suffix)`, `datetime()`.

> **COMMON EXAM TRAP**
> Dynamic data masking is **not** a security boundary. A user who can run queries can often infer masked values with `WHERE` predicates and range probing. It reduces accidental exposure; it does not stop a determined user. If the requirement is "must not be able to determine the value", the answer is CLS, OLS, or OneLake security — not DDM.

4.7 Sensitivity labels and endorsement

Sensitivity labels come from Microsoft Purview, apply to any Fabric item, **flow downstream automatically**, and **persist into exported files** where they enforce encryption.

Endorsement marks trust and affects discovery, not permissions:

Level	Who applies it
Promoted	Anyone with Write on the item
Certified	Only reviewers nominated by the tenant admin
Master data	Same restriction as Certified

4.8 Audit logs

Fabric writes user and system activity to the unified audit log, searchable in the **Microsoft Purview compliance portal** and retrievable programmatically.

Method	Use it for
Track user activities in Fabric	The canonical reference for what is logged and how to get it
Microsoft Purview audit search	Ad-hoc investigation: who viewed, exported, deleted, or shared
`Get-PowerBIActivityEvent` (PowerShell) / Activity Events REST API	Scheduled extraction into a lakehouse for long-term retention and reporting
Metadata scanning APIs	Tenant-wide inventory of items, lineage, and endorsement

Typical exam scenario: *"Management must see who exported data from a semantic model over the last 90 days."* The answer is the audit log — via Purview audit search, or an activity-events extraction pipeline if retention beyond the default window is required.

4.9 Chapter 4 self-check

1. A support engineer must re-run failed pipelines but must not edit them. What do you grant?
2. Credit card numbers must show only the last four digits to analysts, but stay intact for the finance service account. What do you use?
3. The same restriction must apply whether a user queries with Spark, T-SQL, or a Direct Lake model. Which feature?
4. You must report on who deleted a lakehouse three weeks ago. Where do you look?
5. Which endorsement level requires a tenant-admin-nominated reviewer?

> **Answers.** 1. Item-level Execute (plus Read) rather than a workspace role that allows editing. 2. Dynamic data masking with `partial()`, plus `GRANT UNMASK` to the finance principal. 3. OneLake security data access roles. 4. The Fabric/Purview audit log — see track user activities. 5. Certified (and Master data).

4.10 Learn more

- Security in Fabric · permission model · workspace roles
- OneLake security · data access control model
- Dynamic data masking · RLS · CLS
- Information protection · endorsement · track user activities
- Training: Secure data access in Fabric · Secure a Fabric data warehouse · Govern analytics data

---

Chapter 5 — Orchestrate processes

EXAM FOCUS

Orchestrate processes — choose between Dataflow Gen2, a pipeline, and a notebook · design and implement schedules and event-based triggers · implement orchestration patterns with notebooks and pipelines, including parameters and dynamic expressions.

5.1 Choosing the tool

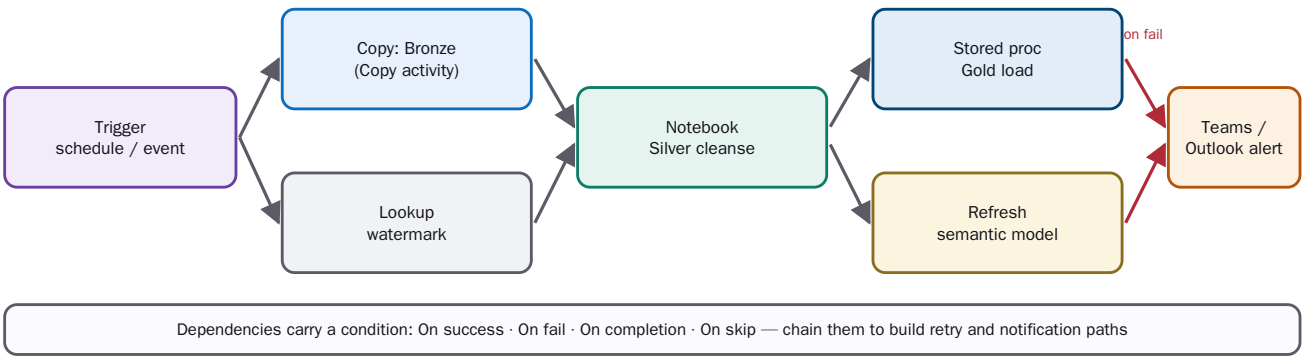
	Dataflow Gen2	Data pipeline	Notebook
Purpose	Transform with Power Query	Orchestrate and move	Transform with code
Language	M	JSON activities + expressions	PySpark, Spark SQL, Scala, R
Control flow	None	**Yes** — conditions, loops, retries, dependencies	Code-level, plus `notebookutils`
Best for	Analyst-built cleansing, many connectors	Scheduling, dependencies, Copy at scale	Complex logic, big data, ML, file work
Weak at	Very large volumes, complex logic	Actual transformation	Being maintained by non-coders

Guidance: choose between pipeline, dataflow, and Spark.

KEY TAKEAWAY

The usual production answer is **all three**: a pipeline orchestrates, a notebook (or dataflow) transforms, and the pipeline handles retries, dependencies, and alerting. Questions that say "orchestrate", "on failure", or "run these in order" are pipeline questions even when the work is done in notebooks.

5.2 Orchestration patterns



A realistic pipeline: lookup, copy, transform, load, refresh, notify — with conditional dependencies on every edge.

Activity dependencies carry a condition: **On success**, **On fail**, **On completion**, **On skip**. Combining them is how you build retry and notification paths. Reference: activity overview.

The control-flow activities you should recognise:

Activity	Use
Lookup	Read a watermark, a config table, or a list of tables to process
ForEach	Iterate over that list — set **batch count** for parallelism, or sequential for ordered loads
If Condition / **Switch**	Branch on an expression
Until	Poll until a condition is met (for example an external job completes)
Copy	Move data at scale, Copy activity
Notebook	Run a Spark notebook with parameters, notebook activity
Dataflow	Run a Dataflow Gen2, dataflow activity
Stored procedure	Call warehouse T-SQL
Script	Run ad-hoc SQL
Semantic model refresh	Refresh or frame a model at the end of the load
Invoke pipeline	Compose a parent/child pipeline hierarchy
Teams / Outlook	Notify on failure
Fail	Deliberately fail the pipeline with a custom message

5.3 Parameters and dynamic expressions

Parameters make one pipeline serve many tables — the **metadata-driven** pattern the exam loves.

```
// Pipeline parameters:  SourceTable (string), LoadType (string)

@pipeline().parameters.SourceTable
@concat('bronze/', pipeline().parameters.SourceTable, '/', formatDateTime(utcNow(),
'yyyy/MM/dd'), '/')
@activity('Lookup watermark').output.firstRow.LastLoadedOn
@equals(pipeline().parameters.LoadType, 'full')
@greater(activity('Copy data').output.rowsCopied, 0)
@{concat('SELECT * FROM ', pipeline().parameters.SourceTable,
         ' WHERE ModifiedDate > ''', activity('Lookup
watermark').output.firstRow.LastLoadedOn, '''')}
```

System variables worth knowing: `@pipeline().RunId`, `@pipeline().TriggerTime`, `@pipeline().parameters.<name>`, `@item()` inside ForEach. Full syntax: expression language.

Passing parameters into a notebook and reading them back out:

```
# In the notebook, a cell tagged "Parameters" declares defaults
table_name = "sales"
load_date  = "2026-09-26"

# ... do the work ...

# Return a value to the calling pipeline
import notebookutils
notebookutils.notebook.exit(f"{rows_written}")
```

Chaining notebooks from a notebook, including a parallel DAG:

```
import notebookutils
# Run one child notebook with a 90-second timeout and parameters
notebookutils.notebook.run("Silver_Cleanse", 90, {"table": "sales"})

# Or run many with dependencies, in parallel where possible
dag = {
    "activities": [
        {"name": "bronze", "path": "Bronze_Load",  "timeoutPerCellInSeconds": 120},
        {"name": "silver", "path": "Silver_Clean", "dependencies": ["bronze"]},
        {"name": "gold",   "path": "Gold_Build",   "dependencies": ["silver"]},
    ]
}
notebookutils.notebook.runMultiple(dag)
```

Reference: notebook utilities and Spark utilities.

5.4 Schedules and event-based triggers

Trigger	What starts the run
Schedule	A recurrence on a pipeline, notebook, or dataflow refresh
Storage event trigger	A blob created or deleted in Azure Storage — storage event triggers
Fabric/OneLake event trigger	A OneLake or job event routed through Activator
Activator rule on a stream	A data condition in an eventstream or KQL table crossing a threshold
Invoke pipeline	A parent pipeline calling a child
API / external	The Fabric REST API called from an external scheduler or Airflow DAG

> **KNOW THE DIFFERENCE**
>
> **Schedule vs event trigger.** A schedule runs whether or not new data arrived — simple, predictable, and it can process nothing. An event trigger runs when something actually happened — lower latency and no empty runs, but it needs the event source wired up and can fire in bursts. "As soon as the file lands" ⇒ event trigger. "Every night at 02:00" ⇒ schedule.

5.5 Chapter 5 self-check

1. Forty tables must be copied with the same logic, and the list changes monthly. What pattern?
2. A load must start within seconds of a file arriving in ADLS Gen2. What do you configure?
3. A notebook must tell the pipeline how many rows it wrote. How?
4. A pipeline must email the on-call engineer when the Gold load fails but still mark the run as failed. How?
5. Which activity waits for an external system to finish before continuing?

> **Answers.** 1. Metadata-driven: a Lookup over a control table feeding a ForEach with a parameterised Copy. 2. A storage event trigger. 3. `notebookutils.notebook.exit(value)`, read by the caller from the notebook activity output. 4. An Outlook/Teams activity on the **On fail** dependency, followed by a **Fail** activity so the run's status stays failed. 5. Until, polling a condition.

5.6 Learn more

- Activity overview · expression language · pipeline runs
- Notebook activity · dataflow activity · notebook utilities
- Storage event triggers · Activator
- Training: Orchestrate processes and data movement

Chapter 6 — Design and implement loading patterns

EXAM FOCUS

Design and implement loading patterns — design and implement full and incremental data loads · prepare data for loading into a dimensional model · design and implement a loading pattern for streaming data.

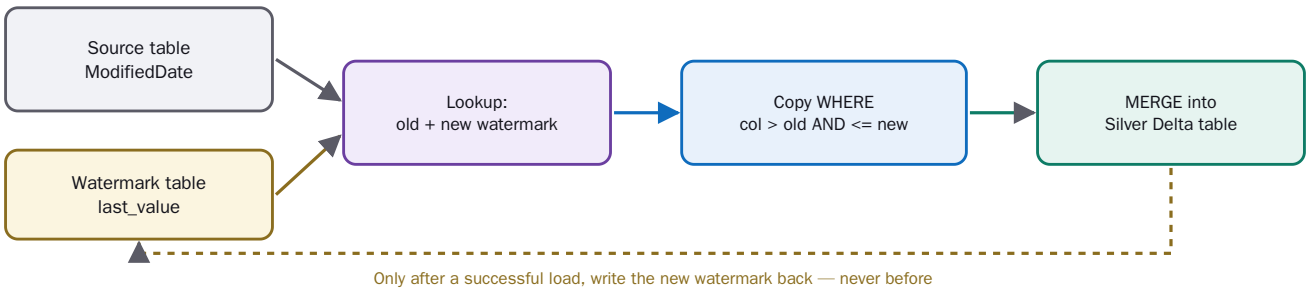
6.1 Full versus incremental

	Full load	Incremental load
What moves	Everything, every run	Only what changed since last run
Target write	`OVERWRITE` / truncate-and-load	`MERGE` / append
Cost and duration	Grows with total size	Grows with change volume
Complexity	Low	Watermarks, CDC, late data, idempotency
Use when	Small tables, dimensions, full restatement, first load	Large facts, frequent runs, narrow change windows

KEY TAKEAWAY

Incremental is not automatically better. For a 50,000-row dimension that changes unpredictably, a full reload is simpler, cheaper to maintain, and always correct. Reserve incremental for tables where the full scan genuinely hurts.

6.2 The watermark pattern



Read the old watermark, copy the delta, merge it, and only then write the new watermark back.

```
-- Control table in the warehouse
CREATE TABLE etl.Watermark (
    TableName     VARCHAR(128) NOT NULL,
    LastLoadedOn  DATETIME2(6) NOT NULL
);
```

The pipeline:

1. **Lookup** the old watermark for this table, and the source's current maximum.
2. **Copy** with a query filtered as `WHERE ModifiedDate > @old AND ModifiedDate <= @new` — a half-open interval so no row is loaded twice or skipped.
3. **Merge** the delta into the Silver table.
4. **Stored procedure** writes the new watermark, *only after* step 3 succeeds.

```
# Step 3 in a notebook, on a Delta table
from delta.tables import DeltaTable

updates = spark.read.table("bronze.sales_delta")
target  = DeltaTable.forName(spark, "silver.sales")

(target.alias("t")
   .merge(updates.alias("s"), "t.SalesOrderId = s.SalesOrderId AND t.LineNumber =
s.LineNumber")
   .whenMatchedUpdateAll()
   .whenNotMatchedInsertAll()
   .execute())
```

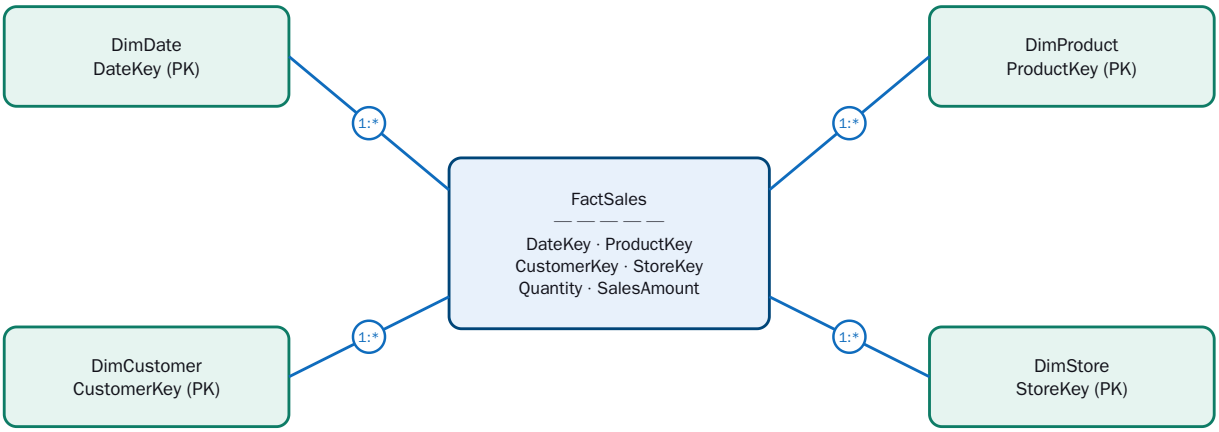
> **COMMON EXAM TRAP**
>
> Writing the watermark **before** the load succeeds is the classic data-loss bug, and it is a favourite exam scenario. The correct order is always: load → verify → commit the watermark. A pipeline that updates the watermark on **On completion** rather than **On success** silently loses rows on every failure.

Alternatives to a watermark column:

- **Change Data Capture / change feed** at the source, for true deletes as well as updates.
- **Delta change data feed** on a Fabric Delta table: `TBLPROPERTIES (delta.enableChangeDataFeed = true)`, then read `table_changes()` to get only the changed rows downstream.
- **Mirroring**, which removes the pattern entirely for supported source databases.
- **Copy job**, which offers incremental copy without building the plumbing yourself.

6.3 Preparing data for a dimensional model

Target shape: a star schema with surrogate keys. See dimensional modeling in Fabric, dimension tables, fact tables, and load tables.



One-to-many, single-direction filters flowing from dimensions to the fact table

Facts in the middle, dimensions around the outside, joined by surrogate keys.

Slowly changing dimensions are the part most often tested:

SCD Type 1 — overwrite, history lost

Key	Customer	City	Start	End	Current
101	Contoso	Osaka	—	—	Yes

City changed Tokyo → Osaka; the old value is gone

SCD Type 2 — new row, history kept

Key	Customer	City	Start	End	Current
101	Contoso	Tokyo	2024-01	2026-03	No
217	Contoso	Osaka	2026-03	9999	Yes

Facts before 2026-03 still join to Tokyo

Type 1 overwrites and forgets. Type 2 adds a row and keeps history, which is what "must report on the value as it was at the time" requires.

Type	Behaviour	When to use
Type 0	Never changes	Immutable attributes such as date of birth
Type 1	Overwrite the old value	Corrections; history has no business meaning
Type 2	Close the old row, insert a new current row with validity dates and an `IsCurrent` flag	History matters: "sales by the customer's region *at the time of sale*"
Type 3	Keep a "previous value" column	Only one prior value ever needed
Type 6	1 + 2 + 3 combined	Rare; know the name

A Type 2 upsert in Spark SQL:

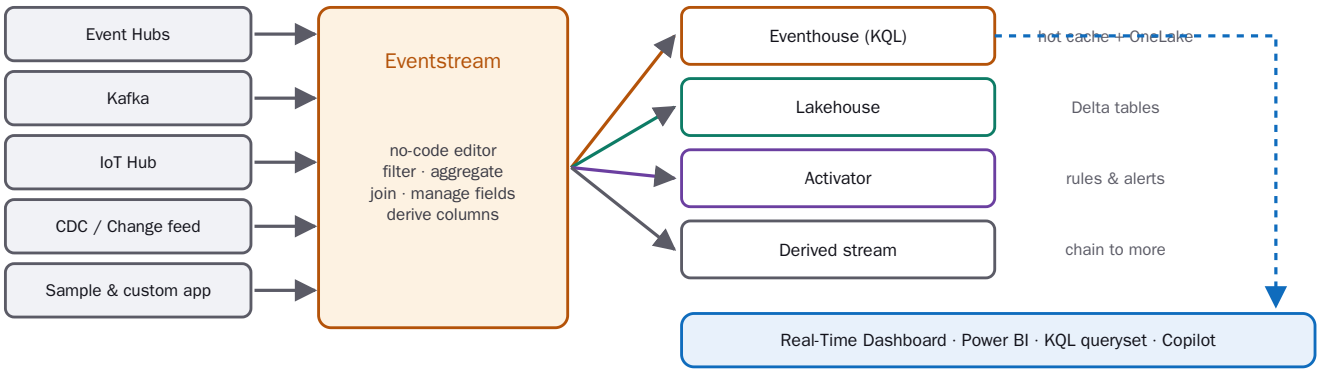
```
-- 1) Close the rows whose tracked attributes changed
MERGE INTO silver.DimCustomer AS t
USING staging.Customer        AS s
   ON t.CustomerId = s.CustomerId AND t.IsCurrent = true
WHEN MATCHED AND (t.City <> s.City OR t.Segment <> s.Segment) THEN
  UPDATE SET t.IsCurrent = false, t.EndDate = current_date();

-- 2) Insert the new current rows (new customers and changed ones)
INSERT INTO silver.DimCustomer
SELECT s.CustomerId, s.Name, s.City, s.Segment, current_date() AS StartDate,
       DATE'9999-12-31' AS EndDate, true AS IsCurrent
FROM   staging.Customer s
LEFT   JOIN silver.DimCustomer t
       ON t.CustomerId = s.CustomerId AND t.IsCurrent = true
WHERE  t.CustomerId IS NULL OR t.City <> s.City OR t.Segment <> s.Segment;
```

Other dimensional preparation tasks:

- **Surrogate keys** — no `IDENTITY` in Fabric Warehouse; generate with `ROW_NUMBER()`, a hash of the business key, or a sequence table.
- **Late-arriving facts** — the fact arrives before its dimension row. Insert an **inferred member** (a placeholder dimension row keyed on the business key) and let the real attributes fill in later.
- **Unknown member** — a `-1` row in every dimension so fact rows with a missing key still join, instead of disappearing from an inner join.
- **Degenerate dimensions** — order numbers stored on the fact with no dimension table of their own.

6.4 Loading patterns for streaming data



Streaming sources land through an eventstream, then fan out to an eventhouse, a lakehouse, an Activator rule, or another stream.

Three broad patterns:

Pattern	How	When
Stream to eventhouse	Eventstream → Eventhouse (KQL)	Sub-second query over high-volume telemetry; keep hot data in cache, cold in OneLake
Stream to lakehouse	Eventstream → lakehouse Delta table, or Spark Structured Streaming	The stream must join batch data and feed the medallion layers
Stream to both	One eventstream with two destinations	Real-time dashboards from KQL, historical analytics from Delta

Points that matter in exam scenarios:

- Streaming writes into Delta produce **many small files**. Schedule `OPTIMIZE`, or use the eventstream's own batching/windowing into the lakehouse.
- Structured Streaming requires a **checkpoint location** for exactly-once semantics — lose it and you reprocess or skip.
- Decide **append vs upsert** early. Append-only Bronze is the safest landing pattern; deduplicate in Silver.
- Handle **late-arriving events** with a watermark in Structured Streaming (`withWatermark`) or with update policies in KQL.

```
(spark.readStream.format("delta").table("bronze.events")
   .withWatermark("EventTime", "10 minutes")
   .groupBy(window("EventTime", "5 minutes"), "DeviceId")
   .agg(avg("Temperature").alias("AvgTemp"))
 .writeStream
   .option("checkpointLocation", "Files/checkpoints/agg_events")
   .outputMode("append")
   .toTable("silver.events_5min"))
```

6.5 Chapter 6 self-check

1. Sales must be reportable by the customer's region as it was on the order date. Which SCD type?
2. A fact row arrives for a product that is not yet in `DimProduct`. What do you insert?
3. What is the correct order of watermark operations?
4. What breaks exactly-once processing in Spark Structured Streaming?
5. A Bronze Delta table has 200,000 files after a week of streaming. What is the fix?

> **Answers.** 1. Type 2. 2. An inferred (placeholder) dimension member, updated with real attributes when they arrive. 3. Read the old watermark → load → verify success → write the new watermark. 4. Losing or changing the checkpoint location. 5. Scheduled `OPTIMIZE` (compaction), plus `VACUUM` after the retention period, and larger micro-batches if possible.

6.6 Learn more

- Dimensional modeling in Fabric · load tables

- Lakehouse and Delta tables · streaming data into a lakehouse
- Copy job · mirroring
- Structured Streaming reference
- Training: Design dimensional models · Ingest data with Microsoft Fabric

---

Chapter 7 — Ingest and transform batch data

EXAM FOCUS

Ingest and transform batch data — choose an appropriate data store · choose between Dataflows Gen2, notebooks, KQL, and T-SQL for data transformation · create and manage OneLake shortcuts · implement mirroring · ingest data by using pipelines · transform data by using PySpark, SQL, and KQL · denormalize data · group and aggregate data · handle duplicate, missing, and late-arriving data.

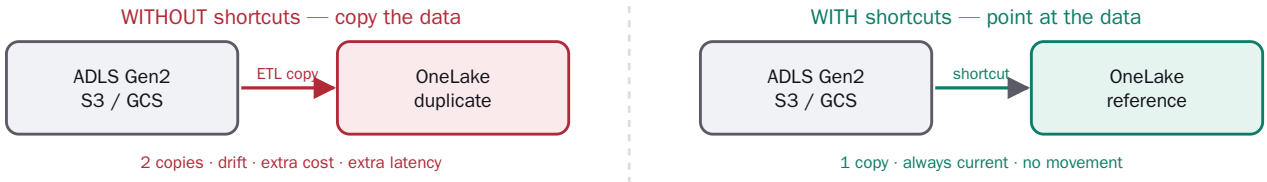
7.1 Choosing the transformation engine

	Dataflow Gen2	Notebook (PySpark)	T-SQL (warehouse)	KQL (eventhouse)
Best at	Connector breadth, low-code cleansing	Any scale, any logic, files	Set-based joins, MERGE, procedures	Time series, telemetry, ingestion-time transforms
Audience	Analysts	Data engineers	SQL developers	Real-time analysts
Writes to	Lakehouse, warehouse, SQL DB, KQL DB	Delta tables, files	Its own warehouse	KQL tables
Cost profile	Higher per GB at large volume	Efficient at large volume	Efficient for set operations	Efficient for append-heavy streams
Limitation	Folding breaks kill performance	Session start-up, code skills	No file access	Not a general ETL engine

COMMON EXAM TRAP

"Choose between Dataflows Gen2, notebooks, KQL, and T-SQL" is a literal exam bullet. Map the *wording* of the requirement: "citizen developer / no code" ⇒ Dataflow Gen2. "Terabytes / custom Python library / unstructured files" ⇒ notebook. "MERGE / stored procedure / multi-table transaction" ⇒ T-SQL warehouse. "Telemetry / last 5 minutes / time series" ⇒ KQL.

7.2 OneLake shortcuts



A shortcut points at data instead of copying it.

Create a shortcut in a lakehouse's **Tables** or **Files** area. Sources: another Fabric lakehouse/warehouse, ADLS Gen2, Amazon S3, S3-compatible storage, Google Cloud Storage, Dataverse, and on-premises

through a gateway.

Consideration	Detail
Where it can live	`Tables/` (must be a valid Delta table) or `Files/` (any folder)
Permissions	Evaluated through the shortcut's connection and the target's OneLake security — the source system's ACLs do **not** automatically flow
Cost	No storage duplication; **egress charges** may apply for cross-cloud reads
Caching	Shortcut caching can reduce repeated egress for external sources
Breaking changes	Deleting the source or changing its path breaks the shortcut; the lakehouse shows an error on read

Reference: create a OneLake shortcut.

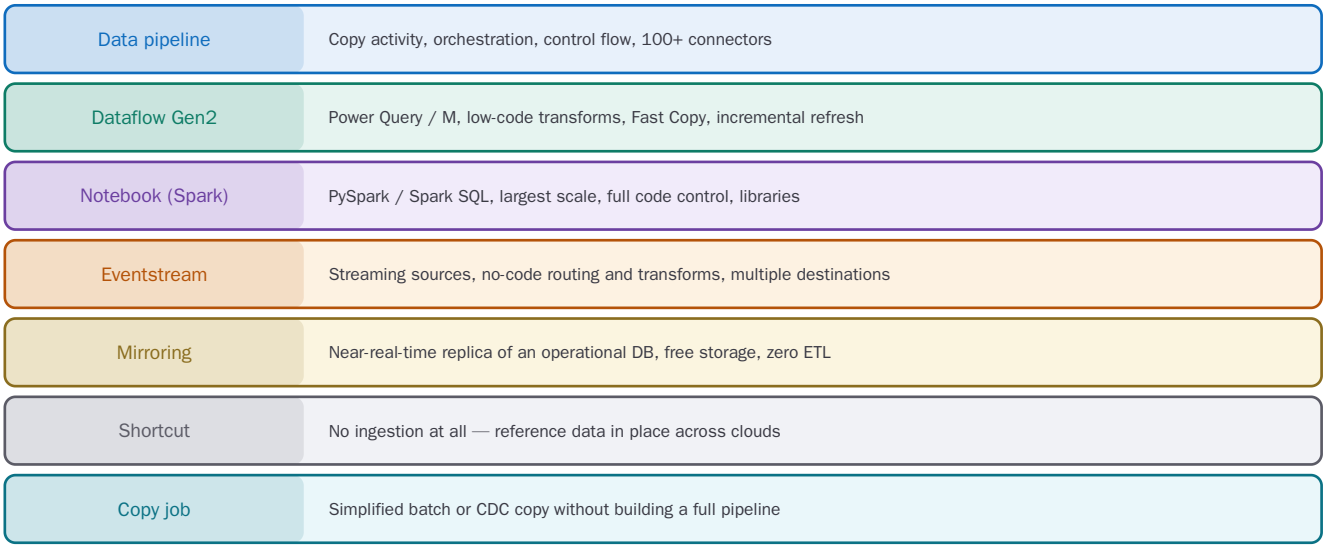
7.3 Mirroring

Mirroring continuously replicates an operational database into OneLake as Delta, with no pipeline to author or maintain.

- Supported sources include Azure SQL Database, Azure SQL Managed Instance, Azure Cosmos DB, Azure Database for PostgreSQL, Snowflake, and Fabric SQL database (mirrored automatically).
- The mirrored data is **read-only** in Fabric and lands as Delta, so Spark, T-SQL, shortcuts, and Direct Lake can all read it.
- Latency is near real time; you do not manage watermarks, CDC plumbing, or schedules.

> **KNOW THE DIFFERENCE**
>
> **Mirroring vs shortcut vs pipeline.** Mirroring = *replicate* a database continuously with no ETL. Shortcut = *reference* files/tables that already sit in a lake. Pipeline = *copy and reshape* on a schedule. "Minimise development effort and keep the analytics copy within seconds of the source" ⇒ mirroring.

7.4 Ingesting with pipelines



Seven ways in. Match the requirement's wording to the tool.

The Copy activity is the workhorse. Settings that come up in questions:

Setting	Why it matters
Source query vs table	A filtered query is how you make a copy incremental
Degree of copy parallelism	Throughput vs source pressure
Staging	Required for some source/sink combinations, and for large bulk loads
Fault tolerance	Skip incompatible rows and log them rather than failing the run
Logging	Session log of skipped rows for later inspection
Partition options	Parallel reads from a partitioned source

Also available: Copy job for simplified batch or CDC copy without building a pipeline, and `COPY INTO`/CTAS inside a warehouse:

```
COPY INTO dbo.FactSales
FROM 'https://<account>.blob.core.windows.net/data/sales/*.parquet'
WITH (FILE_TYPE = 'PARQUET', CREDENTIAL = (IDENTITY = 'Managed Identity'));
```

7.5 Transforming: the same job in three languages

Denormalise, group, and aggregate in PySpark:

```
from pyspark.sql import functions as F

sales = spark.read.table("silver.fact_sales")
prod  = spark.read.table("silver.dim_product")

gold = (sales.join(prod, "ProductKey", "left")
             .groupBy("Category", "OrderDate")
             .agg(F.sum("SalesAmount").alias("SalesAmount"),
                  F.sum("Quantity").alias("Quantity"),
                  F.countDistinct("SalesOrderId").alias("Orders")))

gold.write.mode("overwrite").option("overwriteSchema", "true") \
    .format("delta").saveAsTable("gold.sales_by_category")
```

The same in T-SQL:

```
CREATE TABLE gold.SalesByCategory AS
SELECT p.Category, f.OrderDate,
       SUM(f.SalesAmount)          AS SalesAmount,
       SUM(f.Quantity)             AS Quantity,
       COUNT(DISTINCT f.SalesOrderId) AS Orders
FROM   dbo.FactSales  AS f
LEFT   JOIN dbo.DimProduct AS p ON p.ProductKey = f.ProductKey
GROUP  BY p.Category, f.OrderDate;
```

The same in KQL:

```
FactSales
| lookup kind=leftouter DimProduct on ProductKey
| summarize SalesAmount = sum(SalesAmount),
            Quantity    = sum(Quantity),
            Orders      = dcount(SalesOrderId)
         by Category, bin(OrderDate, 1d)
```

7.6 Duplicate, missing, and late-arriving data

Problem	PySpark	T-SQL	KQL
Exact duplicates	`df.dropDuplicates()`	`SELECT DISTINCT`	`distinct *`
Keep newest per key	`Window.partitionBy("Id").orderBy(desc("Ts"))` + `row_number()==1`	`ROW_NUMBER() OVER (PARTITION BY Id ORDER BY Ts DESC) = 1`	`summarize arg_max(Ts, *) by Id`
Nulls	`fillna()`, `na.drop(subset=[...])`	`COALESCE`, `ISNULL`	`coalesce()`, `isnull()`
Bad types	`to_date`, `try_cast` semantics	`TRY_CAST`	`todatetime()`, `toint()` return null on failure
Late arrivals	`withWatermark` + `MERGE` on the natural key	`MERGE` keyed on business key + event time	Update policy, or ingestion-time transform

Idempotency is the property that makes late and duplicate data survivable: running the same load twice must not double the data. Achieve it with `MERGE` on a business key rather than `INSERT`, and by partitioning writes so a re-run overwrites exactly the partition it owns:

```
(df.write.format("delta")
   .mode("overwrite")
   .option("replaceWhere", "LoadDate = '2026-09-26'")   # overwrite just this partition
   .saveAsTable("bronze.sales"))
```

> **COMMON EXAM TRAP**
>
> Appending in Bronze and deduplicating in Silver is the resilient pattern; deduplicating during ingestion is not. If Bronze is append-only you can always reprocess, and a duplicated source file is corrected in Silver by the `MERGE`. Answers that "prevent duplicates by checking before insert" are usually both slower and wrong under concurrency.

7.7 Chapter 7 self-check

1. A team must analyse Parquet in Amazon S3 alongside Fabric tables, with no copy and minimal effort. What do you build?
2. An Azure SQL Database must be analysable in Fabric within seconds of a change, with no ETL. What do you choose?
3. A source file is occasionally delivered twice. How do you keep the Silver table correct?
4. Which engine transforms telemetry as it is ingested into an eventhouse?
5. A copy run fails on three malformed rows out of 40 million. How do you let it finish and still see them?

> **Answers.** 1. A OneLake shortcut to the S3 location. 2. Mirroring. 3. `MERGE` on the business key (idempotent), with Bronze append-only so reprocessing is safe. 4. KQL, via an update policy on the target table. 5. Enable fault tolerance with skipped-row logging on the Copy activity.

7.8 Learn more

- OneLake shortcuts · create a shortcut · mirroring
- Copy activity · Copy job · connector overview
- Dataflows Gen2 · Fast Copy
- Lakehouse tutorial · ingest data into a warehouse
- Training: Use Apache Spark in Fabric · Ingest with Dataflows Gen2 · Load data into a warehouse

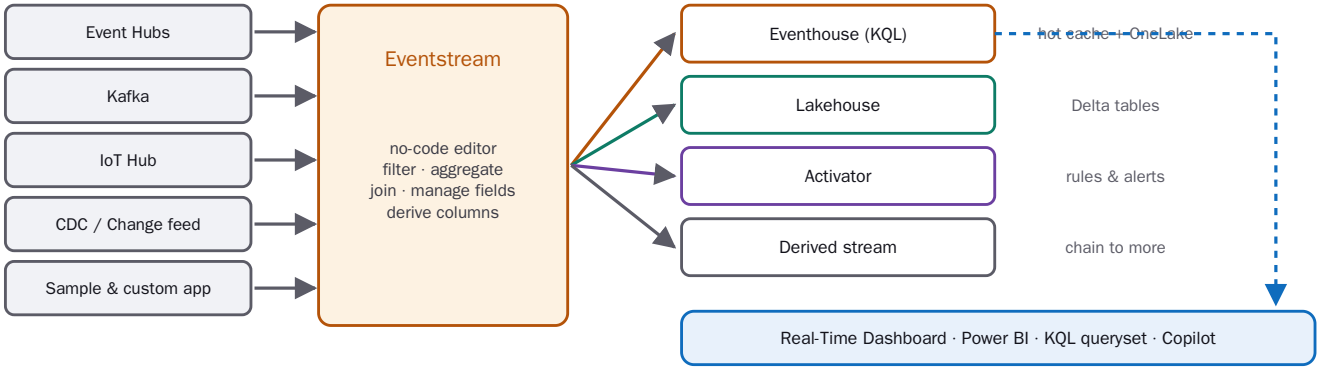
---

Chapter 8 — Ingest and transform streaming data

EXAM FOCUS

Ingest and transform streaming data — choose an appropriate streaming engine · choose between native tables and OneLake shortcuts in Real-Time Intelligence · choose between Query acceleration for OneLake shortcuts and standard OneLake shortcuts · process data by using Eventstream · process data by using Spark structured streaming · process data by using KQL · create windowing functions.

8.1 The Real-Time Intelligence architecture



Sources → Eventstream → destinations. The eventhouse is the query engine; Activator turns conditions into actions.

Component	Role
Real-Time hub	Tenant-wide catalogue of streams and events
Eventstream	No-code ingestion, transformation, and routing
Eventhouse	The container for KQL databases; hot cache plus OneLake
KQL database / queryset	Where the data lands and is queried — create a database
Activator	Rules on streams and reports that trigger alerts or Fabric jobs
Real-Time Dashboard	Low-latency visuals over KQL

8.2 Choosing a streaming engine

Engine	Choose it when
Eventstream	No-code requirement, multiple destinations, simple filter/aggregate/route, fastest to build
Spark Structured Streaming	The stream must join large batch data, use custom Python logic or ML, or write complex Delta merges
KQL update policy / materialized view	The transformation belongs at ingestion time inside the eventhouse, or you need pre-aggregated views maintained automatically

> **KNOW THE DIFFERENCE**
>
> **Eventstream vs Structured Streaming.** Eventstream is a managed, no-code pipeline with built-in sources and sinks — start here. Structured Streaming is code on Spark: more powerful, more control, but you own checkpoints, scaling, and failure handling. "Without writing code" ⇒ Eventstream. "Enrich each event against a 2 TB Delta dimension" ⇒ Structured Streaming.

8.3 Processing with Eventstream

An eventstream has three parts: **sources**, an optional **event processor**, and **destinations**.

- Sources: Azure Event Hubs, Kafka, IoT Hub, CDC feeds from databases, Fabric events, sample data, and custom endpoints — see add an Event Hubs source.
- Processor operations: filter, manage fields, expand, aggregate, group by, union, join, and derived columns — see route events based on content.
- Destinations: eventhouse, lakehouse, derived stream, Activator, or a custom endpoint.

> **WATCH OUT**
>
> Writing a high-rate stream directly to a lakehouse creates a very large number of small Delta files. Use the destination's batching options and schedule `OPTIMIZE` on the target table, or land in an eventhouse with OneLake availability instead.

8.4 Native tables versus OneLake shortcuts in Real-Time Intelligence

	Native KQL table	OneLake shortcut in the eventhouse
Where the data lives	Ingested into the eventhouse, indexed, hot-cached	Stays in OneLake/external storage as Delta
Query speed	Fastest — full indexing and caching	Slower; no KQL indexes over the external files
Ingestion cost	You pay to ingest and store	No ingestion; no second copy
Update policies, materialized views, retention policies	Supported	Not in the same way — the data is external
Best for	High-volume telemetry queried constantly	Joining lake data with telemetry; occasional queries over large history

Query acceleration sits between the two: you keep the shortcut, but the engine caches and indexes a defined window of the external data so KQL queries over it behave much more like a native table. You configure the acceleration policy — including how much history to accelerate — and you pay for that cached window.

> **KNOW THE DIFFERENCE**
>
> **Standard shortcut vs query-accelerated shortcut vs native table.** Standard shortcut = cheapest, slowest, no duplication. Accelerated shortcut = pay for a cached window to get near-native speed on recent data while history stays external. Native table = fastest and fully featured, but the data is ingested and stored twice if it also exists in the lake.

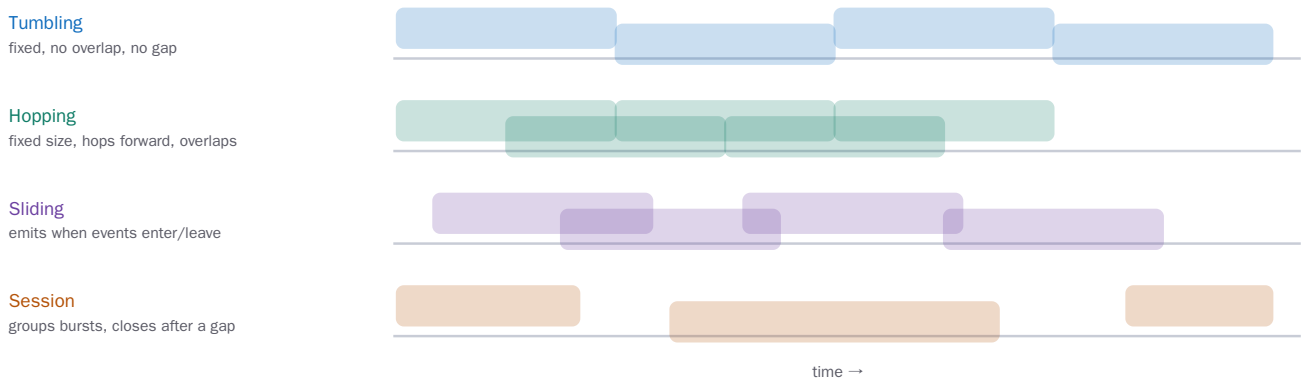
8.5 Processing with KQL inside the eventhouse

An **update policy** transforms rows automatically as they land, writing the result into another table — the KQL equivalent of a Bronze → Silver step at ingestion time:

```
// CleanEvents receives a cleaned, typed projection of every row landing in RawEvents
.alter table CleanEvents policy update @'[{
    "IsEnabled": true,
    "Source": "RawEvents",
    "Query": "RawEvents | where isnotempty(DeviceId)
              | project Timestamp = todatetime(ts), DeviceId, Temp = toreal(temp)",
    "IsTransactional": true
}]'
```

A **materialized view** keeps an aggregate continuously up to date, so dashboards read a small table instead of scanning raw events — see also materialized views in Fabric. Retention, caching, and partitioning policies control how long data is kept and how much stays in hot cache — see table update policy.

8.6 Windowing functions



Four window shapes. The exam tests whether you can pick the right one from a description.

Window	Definition	Give-away wording
Tumbling	Fixed size, no overlap, no gaps. Every event in exactly one window	"every 5 minutes", "per hour, non-overlapping"
Hopping	Fixed size, advances by a hop smaller than the size, so windows overlap	"a 10-minute average, updated every minute"
Sliding	Emits only when an event enters or leaves the window	"whenever the count in the last 5 minutes changes"
Session	Groups events separated by less than a timeout; closes after the gap	"group activity into sessions with a 30-minute idle timeout"
Snapshot	Groups events with exactly the same timestamp	"all events at the same instant"

In KQL, tumbling windows come from `bin()`:

```
Telemetry
| where Timestamp > ago(1h)
| summarize AvgTemp = avg(Temperature), n = count()
      by DeviceId, bin(Timestamp, 5m)     // 5-minute tumbling window
```

In Spark Structured Streaming, `window()` gives tumbling with one argument and hopping with two:

```
from pyspark.sql.functions import window, avg

tumbling = df.groupBy(window("EventTime", "5 minutes"), "DeviceId").agg(avg("Temp"))
hopping  = df.groupBy(window("EventTime", "10 minutes", "1 minute"),
"DeviceId").agg(avg("Temp"))
```

Concept reference for the window shapes: windowing functions.

8.7 Chapter 8 self-check

1. A dashboard needs a rolling 15-minute average refreshed every minute. Which window?

2. You must join every incoming event to a 3 TB dimension table before writing it. Which engine?
3. Telemetry is already in ADLS as Delta and is queried twice a week. Native table or shortcut?
4. The same data is queried constantly with sub-second expectations. What changes?
5. Which eventhouse feature cleans and types rows automatically as they arrive?

> **Answers.** 1. Hopping (size 15 minutes, hop 1 minute). 2. Spark Structured Streaming. 3. A standard OneLake shortcut — no ingestion cost for infrequent queries. 4. Ingest into a native table, or enable query acceleration on the shortcut. 5. An update policy.

8.8 Learn more

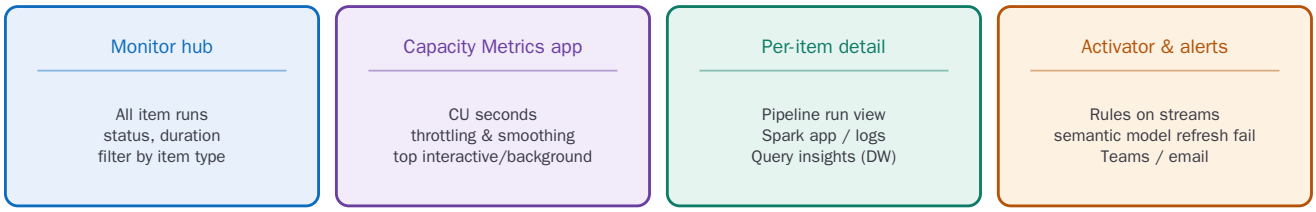
- Real-Time Intelligence overview · Eventhouse · Eventstream
- Query acceleration for shortcuts · materialized views · update policies
- KQL overview · summarize · bin()
- Stream data into a lakehouse
- Training: Implement Real-Time Intelligence · Use Eventstream · Work with real-time data in an Eventhouse · Use Activator

---

Chapter 9 — Monitor Fabric items

> **EXAM FOCUS**
> **Monitor Fabric items** — monitor data ingestion · monitor data transformation · monitor semantic model refresh · configure alerts.

9.1 Where to look



Four monitoring surfaces. Know which question each one answers.

Surface	Answers
Monitoring hub	"What ran, when, for how long, and did it succeed?" across every item type
Capacity Metrics app	"Are we throttled, and what is consuming the CUs?"
Item-level views	Pipeline run detail, Spark application detail, dataflow refresh history, warehouse query insights
Activator	"Tell someone when a condition occurs"

9.2 Monitoring ingestion

For pipelines, open the run from the Monitoring hub and drill into each activity: inputs, outputs, duration, and the Copy activity's **rows read / rows written / throughput / duration by stage**. See pipeline runs and monitor pipeline runs.

What to check when an ingestion is slow rather than failed:

- Rows written versus rows read — a big gap means filtering or rejection.
- Throughput and the parallel copies actually used.
- Whether staging was engaged.
- Whether the source, not Fabric, is the bottleneck (queue time versus transfer time).

For eventstreams, use the built-in monitoring view to see input and output rates per source and destination, plus errors.

9.3 Monitoring transformation

Item	Where
Notebook / Spark job	Spark application detail: jobs, stages, tasks, executors, and logs — see Spark monitoring and detail monitoring
Dataflow Gen2	Refresh history per query, with row counts and error detail
Warehouse T-SQL	Query insights views for historical query text, duration, and frequency
Spark job definition	Same Spark monitoring surfaces as notebooks — Spark job definition

In the Spark UI, the two numbers that explain most slow jobs are **task skew** (one task far longer than the rest) and **shuffle read/write** volume.

9.4 Monitoring semantic model refresh

- Refresh history on the semantic model shows start, end, duration, and error per refresh, including per-table detail for large models.
- For Direct Lake models, what you are watching is **framing**, not row loading.
- A **Semantic model refresh** activity inside the pipeline makes the refresh part of the orchestrated run, so a failed refresh fails the pipeline and triggers your alerting path.

9.5 Configuring alerts

Need	Mechanism
Email or Teams message when a pipeline fails	Outlook/Teams activity on the **On fail** dependency
Alert when a stream crosses a threshold	An Activator rule on the eventstream or KQL table
Alert on a Power BI report condition	An Activator rule driven from the visual
Capacity nearing its limit	Capacity Metrics app plus admin notifications
Custom monitoring and history	Extract run history through the Fabric REST APIs into a lakehouse and build your own report

> **COMMON EXAM TRAP**
>
> An alert that only fires when a run **fails** will never tell you about the load that succeeded while writing zero rows. Production-grade monitoring also asserts on outcomes: row counts written, freshness of the latest partition, and duration against a baseline. Expect a scenario where the pipeline is green but the data is stale.

9.6 Chapter 9 self-check

1. A pipeline succeeds nightly but the Gold table has not changed in three days. What kind of alerting was missing?
2. Where do you find why one Spark stage took 40 minutes?
3. What is a Direct Lake model actually doing during a "refresh"?
4. Which tool tells you whether slowness is caused by capacity throttling?
5. How do you alert when temperature exceeds 80 °C in a live stream?

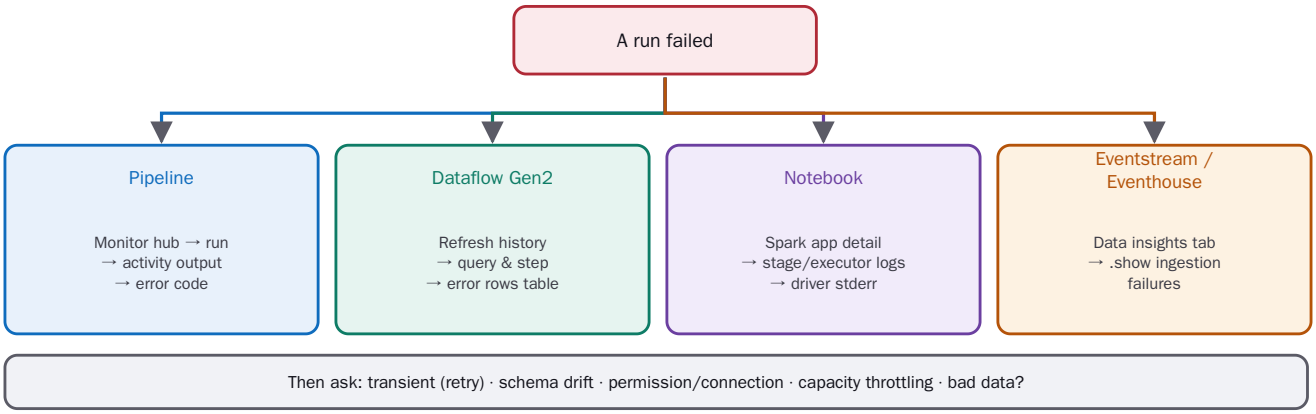
> **Answers.** 1. Outcome-based alerting — row counts or data-freshness checks, not just failure alerts. 2. The Spark application detail view: stages, tasks, and executor logs. 3. Framing — updating metadata to the current Delta version. 4. The Capacity Metrics app. 5. An Activator rule on the eventstream or KQL table.

9.7 Learn more

- Monitoring hub · monitor pipeline runs · pipeline runs
- Apache Spark monitoring · detail monitoring
- Query insights · monitor a warehouse
- Capacity Metrics app · throttling
- Activator · create activators
- Training: Monitor activities in Microsoft Fabric

Chapter 10 — Identify and resolve errors

> **EXAM FOCUS**
> **Identify and resolve errors** — identify and resolve pipeline errors · Dataflow Gen2 errors · notebook errors · Eventhouse errors · Eventstream errors · T-SQL errors · OneLake shortcut errors.



Start from the item type, drill to the detailed log, then classify the cause.

10.1 Pipeline errors

Symptom	Likely cause	Fix
`ErrorCode: 2200` / user configuration error	Bad connection, wrong path, missing file	Verify connection and the resolved dynamic path — check the activity **input** to see what the expression actually produced
Authentication or 401/403	Expired secret, removed service principal, missing gateway	Rotate the credential; prefer workspace identity or a service principal with a managed secret
Sink type conversion failure	Source column type does not fit the target column	Explicit mapping in the Copy activity, or land as string in Bronze and cast in Silver
Timeout	Long-running source query or a busy capacity	Raise the activity timeout, tune the source query, check throttling
Expression error	`@` expression references an activity that did not run	Check dependency conditions; a **Lookup** on a skipped branch yields nothing
Random intermittent failure	Transient network or source throttling	Configure **retry** and **retry interval** on the activity

Always open the activity's **Input** and **Output** JSON in the run detail — most "mystery" pipeline failures are a dynamic expression that resolved to an empty string. See monitor pipeline runs.

10.2 Dataflow Gen2 errors

Symptom	Cause	Fix
Refresh suddenly 10× slower	Query folding broke	Inspect steps with **View native query**; move non-folding steps to the end
"Column not found" after a source change	Schema drift	Avoid hard-coded column indexes; use column names and handle missing columns explicitly
Destination write fails on type	Target schema mismatch	Set the destination's column mapping, or fix types before the destination step
Out of memory / evaluation limit	Too much data through the mashup engine	Enable **staging**, use **Fast Copy**, or move the work to a notebook
Credential error on refresh	Connection owned by a person who left, or expired token	Re-bind the dataflow to a shared connection owned by a service principal

The refresh history shows per-query status and the failing step, which is where you start. Reference: Dataflow Gen2.

10.3 Notebook and Spark errors

Symptom	Cause	Fix
`AnalysisException: Path does not exist`	Wrong relative path or no default lakehouse attached	Attach the lakehouse, or use the full ABFS path
`java.lang.OutOfMemoryError` on a driver	Collecting a big DataFrame with `.collect()` / `toPandas()`	Aggregate first, or write to a table and read back
Executor OOM / spill	Skew, or too-small executors	Repartition on a better key, salt the skewed key, or use larger nodes
One task runs for 40 minutes, the rest finish in 30 seconds	Data skew	Salting, `repartition()`, or a broadcast join
`ModuleNotFoundError`	Library not in the attached environment	Add it to the environment; `%pip install` only lasts for the session
Session takes minutes to start	Custom pool cold start	Use the starter pool, or high concurrency
`ConcurrentModificationException` / Delta conflict	Two writers on one Delta table	Serialise the writes, partition them, or use `replaceWhere` on disjoint partitions

Spark application detail gives you stages, tasks, executors, and logs: Spark monitoring overview.

10.4 Eventhouse and Eventstream errors

Symptom	Cause	Fix
Rows missing after ingestion	Ingestion failures, usually schema or mapping	Run `.show ingestion failures` on the database to see the reason per batch
Wrong column values	Data mapping mismatch between the payload and the table	Fix the ingestion mapping, or clean with an update policy
Ingestion lag	Batching policy waiting for size/time thresholds	Tune the ingestion batching policy for lower latency at the cost of more, smaller batches
Eventstream source disconnected	Consumer group conflict, expired key, or throttled source	Give the eventstream its own consumer group; rotate keys
Destination not receiving data	Filter in the event processor, or a schema mismatch at the destination	Preview the data at each processor node — the editor shows rows flowing through
Duplicated events	At-least-once delivery from the source	Deduplicate downstream on an event ID

Reference: monitor an eventstream.

10.5 T-SQL errors

Error	Meaning	Fix
`Msg 8114` conversion failed	Bad data for the target type	`TRY_CAST` and quarantine the failures
Feature not supported	Not in the Fabric Warehouse T-SQL surface area	Check T-SQL surface area; rewrite, or do it in Spark
`MERGE` fails on the lakehouse SQL endpoint	The endpoint is read-only	Use a warehouse, or Spark `MERGE INTO`
Permission denied	Missing `GRANT`, or RLS/CLS filtering	Check `GRANT`, security policies, and workspace role
Table not found after a Spark write	SQL analytics endpoint metadata has not synced yet	Wait for or trigger the sync; verify the table appears in the endpoint

Troubleshooting reference: troubleshoot Fabric Data Warehouse.

10.6 Shortcut errors

Symptom	Cause	Fix
Shortcut shows an error on read	Source moved, renamed, or deleted	Recreate the shortcut against the new path
Access denied through the shortcut	Connection credential expired or lacks rights at the source	Update the connection; confirm the source-side role assignment
Table shortcut not recognised as a table	Target is not a valid Delta table (for example plain Parquet or CSV)	Put it under `Files/` instead, or convert it to Delta
Data visible in Spark but not in SQL	Endpoint metadata sync, or an unsupported Delta feature	Refresh the endpoint; check Delta reader/writer version compatibility
Unexpected egress cost	Cross-cloud reads on every query	Enable shortcut caching, or copy the data if it is read constantly

10.7 Chapter 10 self-check

1. A copy activity's dynamic file path works in Dev and fails in Prod. What do you inspect first?
2. One Spark task takes 40 minutes; the other 199 take 30 seconds. What is happening?
3. Rows are silently missing from a KQL table after ingestion. Which command?
4. A `MERGE` statement fails against the lakehouse SQL analytics endpoint. Why?
5. A notebook works interactively but fails in a pipeline with `ModuleNotFoundError`. Why?

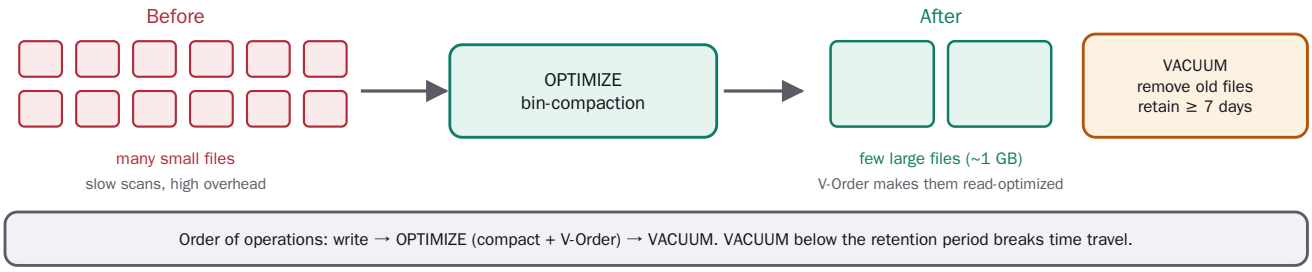
> **Answers.** 1. The activity **Input** JSON, to see what the expression actually resolved to. 2. Data skew — repartition, salt the key, or broadcast the small side. 3. `.show ingestion failures`. 4. The endpoint is read-only; use a warehouse or Spark `MERGE INTO`. 5. The library was installed with `%pip` for the interactive session only; it must be in the attached environment.

Chapter 11 — Optimize performance

EXAM FOCUS

Optimize performance — optimize a lakehouse table · optimize a pipeline · optimize a data warehouse · optimize Eventstream and Eventhouse · optimize Spark performance · optimize query performance.

11.1 Optimise a lakehouse table



Compact, order, then clean up. The "many small files" problem is the single most common lakehouse performance issue.

Technique	What it does	When
`OPTIMIZE`	Bin-compacts many small files into fewer large ones	After streaming or frequent small writes
V-Order	Fabric write optimisation that speeds up reads for Power BI and SQL	On by default; consider off for write-heavy Bronze
`ZORDER BY`	Co-locates related values so predicates skip more files	High-cardinality columns that are filtered often
Partitioning	Physically separates data by a column	Only for large tables with a low-cardinality, frequently filtered column, for example `LoadDate`
`VACUUM`	Deletes files no longer referenced, after the retention period	Regular maintenance; retention below 7 days breaks time travel
Table maintenance UI	Runs OPTIMIZE and VACUUM from the lakehouse	Scheduled housekeeping — table maintenance

```
OPTIMIZE silver.sales ZORDER BY (CustomerId, OrderDate);
VACUUM   silver.sales RETAIN 168 HOURS;
```

COMMON EXAM TRAP

Over-partitioning is worse than not partitioning. Partitioning a 5 GB table by day creates thousands of tiny files and makes everything slower. Rule of thumb: partition only when the table is large (hundreds of GB and up) and each partition would still hold at least around 1 GB.

Reference: Delta optimization and V-Order.

11.2 Optimise Spark

Lever	Effect
Intelligent cache	Automatically caches frequently read files on local SSD
Autotune	Tunes Spark configuration per query from history
Broadcast join	Ships a small dimension to every executor and avoids a shuffle
Avoid Python UDFs	Built-in functions run in the JVM; Python UDFs serialise row by row
`cache()` / `persist()`	Reuse a DataFrame used many times — but only if it fits
Repartition / coalesce	Fix skew and control output file count
Predicate and column pushdown	Select only the columns and rows you need, as early as possible
High concurrency	Shares a session across notebooks, removing start-up cost
Starter pool	Seconds instead of minutes to start

```
from pyspark.sql.functions import broadcast
result = fact.join(broadcast(small_dim), "ProductKey")      # avoids a shuffle
```

11.3 Optimise a pipeline

- Use **Copy activity** rather than a notebook for straight data movement — it is the cheapest path per GB.
- Raise **degree of copy parallelism** and use source **partition options** for large extracts.
- Set **ForEach batch count** to run iterations in parallel when the target can take the concurrency; keep it sequential when order matters.
- Prefer **one pipeline that loads 40 tables in a parameterised loop** over 40 near-identical pipelines.
- Move filtering to the **source query** so you copy less.
- Enable **staging** for sources and sinks that need it, and for very large bulk loads.
- Do not schedule everything at 02:00 — stagger schedules so the capacity is not throttled by a thundering herd.

11.4 Optimise a data warehouse

Technique	Detail
Statistics	The optimiser needs them; Fabric creates and maintains them automatically, but stale or missing statistics on heavily filtered columns are a real cause of bad plans
Result-set caching	Repeated identical queries are served from cache
Right-sized data types	`VARCHAR(50)` instead of `VARCHAR(8000)`; avoid unnecessary `MAX` types
Batch your writes	`COPY INTO` or CTAS rather than thousands of single-row inserts
Avoid `SELECT *`	Columnar storage means unread columns cost nothing — unless you ask for them
Query insights	Find the slowest and most frequent queries before optimising anything
Table clone	Zero-copy clone for testing a change against real data
Time travel	Query a previous state to compare or recover

Full guidance: warehouse performance guidelines and workload management.

11.5 Optimise Eventstream and Eventhouse

Lever	Effect
Ingestion batching policy	Trade latency against the number and size of ingestion batches
Caching (hot) policy	How much recent data stays on fast storage — the main cost/performance dial
Retention policy	How long data is kept at all
Partitioning policy	Improves filtering on high-cardinality keys for very large tables
Materialized views	Pre-aggregate so dashboards read small results
Update policies	Shape data at ingestion instead of in every query
Query acceleration	Speeds up KQL over OneLake shortcuts for a defined window
Eventstream batching to lakehouse	Fewer, larger Delta files at the destination

11.6 Optimise query performance

1. **Measure first** — Query insights for T-SQL, Spark UI for notebooks, `.show queries` for KQL.

2. **Reduce data touched** — filter early, project fewer columns, use partitions and Z-order so files can be skipped.
3. **Avoid row-by-row work** — set-based SQL, DataFrame operations, and KQL operators instead of loops and UDFs.
4. **Pre-compute what is queried repeatedly** — aggregate tables, materialized views, Gold tables.
5. **Cache what is read repeatedly** — result-set caching, intelligent cache, hot cache policy.
6. **Check the platform** — if everything is slow at once, look at the Capacity Metrics app before rewriting any query.

11.7 Chapter 11 self-check

1. A streaming Bronze table has 400,000 tiny files. Which two commands, in which order?
2. A 4 GB dimension table is partitioned by day and queries got slower. Why?
3. A join between a 2 billion-row fact and a 5,000-row dimension shuffles heavily. What do you change?
4. Identical warehouse queries run repeatedly all afternoon. What helps most?
5. A KQL dashboard scans raw events every refresh. What do you build?

> **Answers.** 1. `OPTIMIZE` first (compact), then `VACUUM` after the retention period. 2. Over-partitioning — thousands of tiny files and per-file overhead. 3. A broadcast join on the small dimension. 4. Result-set caching. 5. A materialized view holding the aggregate.

11.8 Learn more

- Delta optimization and V-Order · table maintenance
- Intelligent cache · autotune · Spark compute
- Warehouse performance guidelines · statistics · caching · query insights
- Query acceleration · materialized views
- Capacity throttling · Capacity Metrics app

Chapter 12 — The hands-on lab track

DP-700 questions are written from operational experience. Work through these free guided exercises in order.

#	Lab	Skill area it proves
1	Introduction to end-to-end analytics using Fabric	Platform, workspaces, items
2	Get started with lakehouses	Lakehouse, OneLake, SQL endpoint
3	Use Apache Spark in Microsoft Fabric	Notebooks, DataFrames, Spark settings
4	Work with Delta Lake tables	Delta, time travel, OPTIMIZE, VACUUM
5	Organize a lakehouse using medallion architecture	Bronze/Silver/Gold
6	Ingest data with Dataflows Gen2	Dataflow Gen2, destinations, folding
7	Orchestrate processes and data movement	Pipelines, Copy activity, parameters
8	Get started with data warehouses	Warehouse, T-SQL
9	Load data into a warehouse	COPY INTO, CTAS, stored procedures
10	Design dimensional models in Fabric	Star schema, surrogate keys, SCD
11	Get started with Real-Time Intelligence	Eventhouse, KQL
12	Use Eventstream in Microsoft Fabric	Streaming ingestion and routing
13	Work with real-time data in an Eventhouse	KQL queries, update policies
14	Create Real-Time Dashboards	Serving real-time data
15	Use Activator in Microsoft Fabric	Event-driven alerts and triggers
16	Secure data access in Fabric	Workspace, item, OneLake security
17	Secure a Fabric data warehouse	RLS, CLS, dynamic data masking
18	Govern analytics data in Fabric	Labels, endorsement, domains
19	Implement CI/CD in Microsoft Fabric	Git, deployment pipelines
20	Monitor activities in Microsoft Fabric	Monitoring hub, alerts
21	Monitor a Fabric data warehouse	Query insights, troubleshooting

End-to-end tutorials: lakehouse · Data Factory · Real-Time Intelligence · warehouse.

HANDS-ON LAB

Build this once without following instructions. A pipeline that: looks up a watermark → copies changed rows from a source into Bronze → runs a notebook that merges into Silver with SCD Type 2 → runs a stored procedure that builds a Gold aggregate → refreshes a semantic model → emails you on failure → writes the new watermark on success. If you can build that from memory, you are ready.

Chapter 13 — Practice questions

1. A notebook must process 2 TB daily and join to a 4 KB lookup table. Which join strategy?
2. Which workspace Spark setting removes the two-to-three minute session start-up during development?
3. Data must be restricted identically whether read by Spark, T-SQL, or Direct Lake. Which feature?
4. A pipeline must start within seconds of a file landing in ADLS Gen2. What do you configure?
5. A watermark-based load lost rows after a failed run. What was configured wrongly?
6. Which SCD type is required for "sales by the customer's region at the time of the sale"?
7. Analysts must see only the last four digits of a card number, while a service account sees all of it. Which feature?
8. Telemetry sits in ADLS as Delta and is queried twice a week from an eventhouse. Native table or shortcut?
9. The same telemetry becomes a constantly refreshed dashboard. What changes?
10. A dashboard needs a 15-minute average updated every minute. Which window type?
11. A streaming Bronze table has 400,000 small files. Which maintenance, and in what order?
12. Forty source tables need identical logic, and the list changes monthly. Which pattern?
13. A notebook fails in a pipeline with `ModuleNotFoundError` but works interactively. Why?
14. Which command shows why rows never appeared in a KQL table?
15. A nightly pipeline is green but the Gold table has not changed for three days. What is missing?
16. Which Fabric feature replicates an Azure SQL Database into OneLake with no ETL?
17. A notebook in Production still writes to the Development lakehouse. What do you configure?
18. One Spark task takes 40 minutes while 199 finish in 30 seconds. What is the cause?
19. A 4 GB table partitioned by day got slower after partitioning. Why?
20. Who can mark an item as **Certified**?

▸ Answers and explanations

1. **Broadcast join.** The 4 KB table is shipped to every executor, eliminating the shuffle.
2. **Use the starter pool** (pre-warmed) instead of a custom pool; high concurrency also helps for many small notebooks.
3. **OneLake security data access roles** — enforced at the lake layer for every engine, unlike SQL `GRANT`.
4. **A storage event trigger** on the ADLS container.
5. **The watermark was written on "On completion" (or before the load).** It must be written only after the load succeeds.
6. **Type 2** — a new row with validity dates preserves the historical attribute.
7. **Dynamic data masking** with `partial()`, plus `GRANT UNMASK` to the service account.
8. **A standard OneLake shortcut** — no ingestion cost for occasional queries.
9. **Ingest into a native KQL table, or enable query acceleration** on the shortcut for the hot window.

10. **Hopping** — window size 15 minutes, hop 1 minute.
11. `OPTIMIZE` **first to compact, then** `VACUUM` once past the retention period.
12. **Metadata-driven**: a control table read by Lookup, iterated with ForEach, copied by a parameterised Copy activity.
13. `%pip install` **only applies to the interactive session.** Put the library in the attached environment.
14. `.show ingestion failures`**.**
15. **Outcome-based alerting** — row-count or freshness assertions, not just failure alerts.
16. **Mirroring.**
17. **A default lakehouse deployment rule, or a variable library** referenced by the notebook.
18. **Data skew** on the partition key — salt the key, repartition, or broadcast the small side.
19. **Over-partitioning**: thousands of tiny files and per-file overhead outweigh any pruning benefit at 4 GB.
20. **Only reviewers nominated by the Fabric tenant administrator** (the same restriction applies to Master data).

Chapter 14 — Exam-day checklist and quick reference

14.1 The week before

- Take the DP-700 practice assessment and review every wrong answer against the documentation.
- Re-read the change log in the official study guide — it lists exactly what moved in the latest skills-outline update.
- Watch the relevant Exam Readiness Zone sessions.
- Build the end-to-end pipeline described in the Chapter 12 lab callout, from memory.
- Open the Monitoring hub, a Spark application detail page, and the Capacity Metrics app at least once so you recognise them from a screenshot.

14.2 The 60-second memory dump

Topic	The one line
Workspace roles	Admin > Member > Contributor > Viewer; Viewer cannot run or read files
Item permissions	Read · ReadData (SQL) · ReadAll (Spark/OneLake) · Execute · Build
Security scope	OneLake security = every engine; SQL GRANT = SQL endpoint only
DDM	Obfuscates at query time; `GRANT UNMASK`; not a real boundary
Starter vs custom pool	Seconds vs minutes to start
High concurrency	Shares one session across notebooks; same user, same workspace
Environment	Where libraries and Spark config belong; `%pip` is session-only
Git vs deployment pipeline	History and review vs promotion between stages
Deployment gotcha	Re-bind default lakehouse per stage (rule or variable library)
Watermark order	Load → verify → write watermark. Never on "On completion"
SCD	Type 1 overwrite · Type 2 new row with validity dates
Idempotency	`MERGE` on a business key; `replaceWhere` per partition
Shortcut vs mirroring vs copy	Reference · replicate · duplicate
Native table vs shortcut (RTI)	Ingested and indexed vs queried in place; query acceleration in between
Windows	Tumbling (no overlap) · Hopping (overlap) · Sliding (on change) · Session (gap)
Structured Streaming	Checkpoint location = exactly-once
Small files	`OPTIMIZE` then `VACUUM`; do not over-partition
V-Order	Read optimisation; consider off for write-heavy Bronze
Skew	One long task — salt, repartition, or broadcast
Monitoring	Monitoring hub (runs) · Capacity Metrics (CUs) · Activator (alerts)
Ingestion failures (KQL)	`.show ingestion failures`
Pipeline debugging	Read the activity **Input** JSON first

14.3 Reading exam questions

- Read the **final sentence first** — it tells you what is being asked.
- Watch for **"minimise development effort"** (favours mirroring, shortcuts, Copy job, variable libraries), **"without writing code"** (Eventstream, Dataflow Gen2), and **"regardless of the tool"** (OneLake security).

- **"Near real time"** rarely means streaming infrastructure — mirroring or an event trigger is often the intended answer.
- **"Least privilege"** favours Viewer plus one item permission over a workspace role.
- In case studies, read the requirements list first, then mine the narrative only for what those requirements need.
- Flag and move on; every question carries equal weight.

14.4 Glossary

Term	Meaning
Capacity (F SKU)	The purchased compute pool workspaces run on
CU	Capacity Unit; consumption is smoothed over time
Throttling	Delay applied when sustained usage exceeds the capacity
OneLake	The tenant-wide logical data lake, Delta Parquet
Shortcut	A pointer to data elsewhere, read in place
Mirroring	Managed near-real-time replication of a database into OneLake
Delta Lake	Parquet plus a transaction log: ACID, time travel, schema enforcement
V-Order	Fabric write optimisation that accelerates reads
OPTIMIZE / VACUUM	Compact small files / remove unreferenced files after retention
Z-order	Co-locating related values to improve file skipping
Medallion	Bronze → Silver → Gold layering
Watermark	The stored high-water mark that makes a load incremental
SCD	Slowly changing dimension; Type 1 overwrites, Type 2 keeps history
Inferred member	Placeholder dimension row created when a fact arrives first
Idempotent	Re-running produces the same result, no duplicates
Structured Streaming	Spark's streaming engine; requires a checkpoint location
Eventstream	No-code streaming ingestion, transformation, and routing
Eventhouse / KQL database	The Real-Time Intelligence store queried with KQL
Update policy	KQL rule that transforms rows at ingestion into another table
Materialized view	Continuously maintained aggregate in an eventhouse
Query acceleration	Caching/indexing of a OneLake shortcut for faster KQL
Activator	Rules engine that turns data conditions into alerts or jobs
Monitoring hub	The cross-item view of runs and their status
Environment	Reusable Spark libraries and configuration
High concurrency	Multiple notebooks sharing one Spark session
Variable library	Named values resolved per deployment stage
Database project	Declarative warehouse schema, built to a dacpac

14.5 Where to go next

- **Renew** every 12 months with the free renewal assessment.
- **Pair with DP-600**, the Fabric Analytics Engineer Associate certification, for semantic modelling and Direct Lake. The DP-600 guide in this series follows the same structure.
- Track changes: what's new in Fabric and the Microsoft Fabric blog.
- Ask questions in the Fabric Community and Microsoft Q&A.

> **KEY TAKEAWAY**
>
> For every scenario, be able to answer in one sentence: *which store, which ingestion tool, which orchestration trigger, which security layer, and how I would know it broke.* That is what 700 points looks like.

Appendix A — One-page decision tables

A.1 Which ingestion mechanism?

Signal in the question	Answer
"Orchestrate", "dependencies", "on failure", "retry"	Data pipeline
"No code", "analyst", "Power Query"	Dataflow Gen2
"Custom library", "terabytes", "complex logic", "files"	Notebook (Spark)
"Continuous events", "route to several destinations"	Eventstream
"Near real time replica", "no ETL to maintain"	Mirroring
"Do not copy", "data stays in S3/ADLS"	Shortcut
"Simple batch or CDC copy, minimal setup"	Copy job

A.2 Which transformation engine?

Signal	Answer
"Citizen developer", "low code"	Dataflow Gen2
"MERGE", "stored procedure", "multi-table transaction"	Warehouse T-SQL
"Machine learning", "unstructured", "very large scale"	Notebook (PySpark)
"Telemetry", "at ingestion time", "time series"	KQL update policy / materialized view

A.3 Which trigger?

Signal	Answer
"Every night at 02:00"	Schedule
"As soon as the file arrives"	Storage event trigger
"When a value crosses a threshold"	Activator rule
"After the upstream pipeline finishes"	Invoke pipeline, or dependency within one pipeline

A.4 Which fix for slowness?

Signal	Answer
"Thousands of small files"	`OPTIMIZE`, then `VACUUM`
"One task much slower than the rest"	Skew — salt, repartition, or broadcast
"Repeated identical SQL queries"	Result-set caching
"Dashboard scans raw events"	Materialized view
"Sessions take minutes to start"	Starter pool / high concurrency
"Everything slowed down at once"	Capacity throttling — check the Metrics app
"KQL over a shortcut is slow"	Query acceleration, or ingest into a native table

A.5 Which security control?

Signal	Answer
"Whatever tool they use"	OneLake security
"Only through SQL"	`GRANT` / `DENY`, RLS, CLS
"Show only the last four digits"	Dynamic data masking
"The column must not exist for them"	Object-level security
"Protect the exported file"	Sensitivity label
"Who deleted it, three weeks ago"	Audit logs

Appendix B — Every Microsoft Learn link in this guide

Every link below already carries the Student Ambassadors Contributor ID `wt.mc_id=studentamb_657938`. Share any of them directly.

Resource	URL (includes Contributor ID)
Microsoft Certified: Fabric Data Engineer Associate	https://learn.microsoft.com/credentials/certifications/fabric-data-engineer-associate/?wt.mc_id=studentamb_657938
official DP-700 study guide	https://learn.microsoft.com/credentials/certifications/resources/study-guides/dp-700?wt.mc_id=studentamb_657938
exam duration and experience	https://learn.microsoft.com/credentials/support/exam-duration-exam-experience?wt.mc_id=studentamb_657938
exam scoring and score reports	https://learn.microsoft.com/credentials/certifications/exam-scoring-reports?wt.mc_id=studentamb_657938
retake policy	https://learn.microsoft.com/credentials/support/retake-policy?wt.mc_id=studentamb_657938
renew your certification	https://learn.microsoft.com/credentials/certifications/renew-your-microsoft-certification?wt.mc_id=studentamb_657938
Request accommodations	https://learn.microsoft.com/credentials/certifications/request-accommodations?wt.mc_id=studentamb_657938
DP-700 practice assessment	https://learn.microsoft.com/credentials/certifications/fabric-data-engineer-associate/practice/assessment?assessment-type=practice&assessmentId=1704375541&practice-assessment-type=certification&wt.mc_id=studentamb_657938
DP-700 exam page	https://learn.microsoft.com/credentials/certifications/exams/dp-700/?wt.mc_id=studentamb_657938
Get started with Microsoft Fabric	https://learn.microsoft.com/training/paths/get-started-fabric/?wt.mc_id=studentamb_657938
Implement a Lakehouse	https://learn.microsoft.com/training/paths/implement-lakehouse-microsoft-fabric/?wt.mc_id=studentamb_657938
Ingest data with Microsoft Fabric	https://learn.microsoft.com/training/paths/ingest-data-with-microsoft-fabric/?wt.mc_id=studentamb_657938
Implement Real-Time Intelligence	https://learn.microsoft.com/training/paths/explore-real-time-analytics-microsoft-fabric/?wt.mc_id=studentamb_657938
Manage a Fabric environment	https://learn.microsoft.com/training/paths/manage-microsoft-fabric-environment/?wt.mc_id=studentamb_657938
Secure and govern analytics data	https://learn.microsoft.com/training/paths/secure-govern-analytics-data/?wt.mc_id=studentamb_657938
Monitor activities in Fabric	https://learn.microsoft.com/training/modules/monitor-fabric-items/?wt.mc_id=studentamb_657938
Exam Readiness Zone	https://learn.microsoft.com/shows/exam-readiness-zone/?wt.mc_id=studentamb_657938
Microsoft Fabric trial	https://learn.microsoft.com/fabric/fundamentals/fabric-trial?wt.mc_id=studentamb_657938
Microsoft Fabric	https://learn.microsoft.com/fabric/fundamentals/microsoft-fabric-overview?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
OneLake	https://learn.microsoft.com/fabric/onelake/onelake-overview?wt.mc_id=studentamb_657938
throttling	https://learn.microsoft.com/fabric/enterprise/throttling?wt.mc_id=studentamb_657938
Admin portal	https://learn.microsoft.com/fabric/admin/admin-center?wt.mc_id=studentamb_657938
Capacity settings	https://learn.microsoft.com/fabric/admin/capacity-settings?wt.mc_id=studentamb_657938
scale capacity	https://learn.microsoft.com/fabric/enterprise/scale-capacity?wt.mc_id=studentamb_657938
Domains	https://learn.microsoft.com/fabric/governance/domains?wt.mc_id=studentamb_657938
Workspaces	https://learn.microsoft.com/fabric/fundamentals/workspaces?wt.mc_id=studentamb_657938
paused and resumed	https://learn.microsoft.com/fabric/enterprise/pause-resume?wt.mc_id=studentamb_657938
Capacity Metrics app	https://learn.microsoft.com/fabric/enterprise/metrics-app?wt.mc_id=studentamb_657938
Medallion	https://learn.microsoft.com/fabric/onelake/onelake-medallion-lakehouse-architecture?wt.mc_id=studentamb_657938
choose a data store	https://learn.microsoft.com/fabric/fundamentals/decision-guide-data-store?wt.mc_id=studentamb_657938
pipeline vs dataflow vs Spark	https://learn.microsoft.com/fabric/fundamentals/decision-guide-pipeline-dataflow-spark?wt.mc_id=studentamb_657938
Delta table	https://learn.microsoft.com/fabric/data-engineering/lakehouse-and-delta-tables?wt.mc_id=studentamb_657938
Delta optimization and V-Order	https://learn.microsoft.com/fabric/data-engineering/delta-optimization-and-v-order?wt.mc_id=studentamb_657938
Spark workspace administration settings	https://learn.microsoft.com/fabric/data-engineering/workspace-admin-settings?wt.mc_id=studentamb_657938
Spark compute	https://learn.microsoft.com/fabric/data-engineering/spark-compute?wt.mc_id=studentamb_657938
starter pools	https://learn.microsoft.com/fabric/data-engineering/configure-starter-pools?wt.mc_id=studentamb_657938
Fabric Runtime	https://learn.microsoft.com/fabric/data-engineering/runtime?wt.mc_id=studentamb_657938
environments	https://learn.microsoft.com/fabric/data-engineering/create-and-use-environment?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
high concurrency	https://learn.microsoft.com/fabric/data-engineering/high-concurrency-overview?wt.mc_id=studentamb_657938
library management	https://learn.microsoft.com/fabric/data-engineering/library-management?wt.mc_id=studentamb_657938
OneLake catalog	https://learn.microsoft.com/fabric/governance/onelake-catalog-overview?wt.mc_id=studentamb_657938
OneLake security	https://learn.microsoft.com/fabric/onelake/onelake-security?wt.mc_id=studentamb_657938
Apache Airflow jobs	https://learn.microsoft.com/fabric/data-factory/apache-airflow-jobs-concepts?wt.mc_id=studentamb_657938
create an Apache Airflow job	https://learn.microsoft.com/fabric/data-factory/create-apache-airflow-jobs?wt.mc_id=studentamb_657938
Git integration	https://learn.microsoft.com/fabric/cicd/git-integration/intro-to-git-integration?wt.mc_id=studentamb_657938
notebook source control and deployment	https://learn.microsoft.com/fabric/data-engineering/notebook-source-control-deployment?wt.mc_id=studentamb_657938
SQL database project	https://learn.microsoft.com/fabric/data-warehouse/source-control?wt.mc_id=studentamb_657938
Deployment pipelines	https://learn.microsoft.com/fabric/cicd/deployment-pipelines/intro-to-deployment-pipelines?wt.mc_id=studentamb_657938
Variable library	https://learn.microsoft.com/fabric/cicd/variable-library/variable-library-overview?wt.mc_id=studentamb_657938
CI/CD overview	https://learn.microsoft.com/fabric/cicd/cicd-overview?wt.mc_id=studentamb_657938
CI/CD best practices	https://learn.microsoft.com/fabric/cicd/best-practices-cicd?wt.mc_id=studentamb_657938
get started	https://learn.microsoft.com/fabric/cicd/deployment-pipelines/get-started-with-deployment-pipelines?wt.mc_id=studentamb_657938
Implement CI/CD in Microsoft Fabric	https://learn.microsoft.com/training/modules/implement-cicd-in-fabric/?wt.mc_id=studentamb_657938
Workspace roles	https://learn.microsoft.com/fabric/fundamentals/roles-workspaces?wt.mc_id=studentamb_657938
Fabric permission model	https://learn.microsoft.com/fabric/security/permission-model?wt.mc_id=studentamb_657938
get started with OneLake security	https://learn.microsoft.com/fabric/onelake/security/get-started-security?wt.mc_id=studentamb_657938
data access control model	https://learn.microsoft.com/fabric/onelake/security/data-access-control-model?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
get started with data access roles	https://learn.microsoft.com/fabric/onelake/security/get-started-data-access-roles?wt.mc_id=studentamb_657938
row-level security	https://learn.microsoft.com/fabric/data-warehouse/row-level-security?wt.mc_id=studentamb_657938
column-level security	https://learn.microsoft.com/fabric/data-warehouse/column-level-security?wt.mc_id=studentamb_657938
SQL granular permissions	https://learn.microsoft.com/fabric/data-warehouse/sql-granular-permissions?wt.mc_id=studentamb_657938
Dynamic data masking	https://learn.microsoft.com/fabric/data-warehouse/dynamic-data-masking?wt.mc_id=studentamb_657938
Sensitivity labels	https://learn.microsoft.com/fabric/governance/information-protection?wt.mc_id=studentamb_657938
Endorsement	https://learn.microsoft.com/fabric/governance/endorsement-overview?wt.mc_id=studentamb_657938
Track user activities in Fabric	https://learn.microsoft.com/fabric/admin/track-user-activities?wt.mc_id=studentamb_657938
Metadata scanning APIs	https://learn.microsoft.com/fabric/governance/metadata-scanning-overview?wt.mc_id=studentamb_657938
Security in Fabric	https://learn.microsoft.com/fabric/security/security-overview?wt.mc_id=studentamb_657938
Secure data access in Fabric	https://learn.microsoft.com/training/modules/secure-data-access-in-fabric/?wt.mc_id=studentamb_657938
Secure a Fabric data warehouse	https://learn.microsoft.com/training/modules/secure-data-warehouse-in-microsoft-fabric/?wt.mc_id=studentamb_657938
Govern analytics data	https://learn.microsoft.com/training/modules/fabric-govern-analytics-data/?wt.mc_id=studentamb_657938
activity overview	https://learn.microsoft.com/fabric/data-factory/activity-overview?wt.mc_id=studentamb_657938
Copy activity	https://learn.microsoft.com/fabric/data-factory/copy-data-activity?wt.mc_id=studentamb_657938
notebook activity	https://learn.microsoft.com/fabric/data-factory/notebook-activity?wt.mc_id=studentamb_657938
dataflow activity	https://learn.microsoft.com/fabric/data-factory/dataflow-activity?wt.mc_id=studentamb_657938
expression language	https://learn.microsoft.com/fabric/data-factory/expression-language?wt.mc_id=studentamb_657938
notebook utilities	https://learn.microsoft.com/fabric/data-engineering/notebook-utilities?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
Spark utilities	https://learn.microsoft.com/fabric/data-engineering/microsoft-spark-utilities?wt.mc_id=studentamb_657938
storage event triggers	https://learn.microsoft.com/fabric/data-factory/pipeline-storage-event-triggers?wt.mc_id=studentamb_657938
Activator	https://learn.microsoft.com/fabric/real-time-intelligence/data-activator/activator-introduction?wt.mc_id=studentamb_657938
pipeline runs	https://learn.microsoft.com/fabric/data-factory/pipeline-runs?wt.mc_id=studentamb_657938
Orchestrate processes and data movement	https://learn.microsoft.com/training/modules/use-data-factory-pipelines-fabric/?wt.mc_id=studentamb_657938
Mirroring	https://learn.microsoft.com/fabric/database/mirrored-database/overview?wt.mc_id=studentamb_657938
Copy job	https://learn.microsoft.com/fabric/data-factory/copy-job?wt.mc_id=studentamb_657938
dimensional modeling in Fabric	https://learn.microsoft.com/fabric/data-warehouse/dimensional-modeling-overview?wt.mc_id=studentamb_657938
dimension tables	https://learn.microsoft.com/fabric/data-warehouse/dimensional-modeling-dimension-tables?wt.mc_id=studentamb_657938
fact tables	https://learn.microsoft.com/fabric/data-warehouse/dimensional-modeling-fact-tables?wt.mc_id=studentamb_657938
load tables	https://learn.microsoft.com/fabric/data-warehouse/dimensional-modeling-load-tables?wt.mc_id=studentamb_657938
streaming data into a lakehouse	https://learn.microsoft.com/fabric/data-engineering/lakehouse-streaming-data?wt.mc_id=studentamb_657938
Structured Streaming reference	https://learn.microsoft.com/azure/databricks/structured-streaming/?wt.mc_id=studentamb_657938
Design dimensional models	https://learn.microsoft.com/training/modules/design-dimensional-models-fabric/?wt.mc_id=studentamb_657938
shortcut	https://learn.microsoft.com/fabric/onelake/onelake-shortcuts?wt.mc_id=studentamb_657938
create a OneLake shortcut	https://learn.microsoft.com/fabric/onelake/create-onelake-shortcut?wt.mc_id=studentamb_657938
Azure SQL Database	https://learn.microsoft.com/fabric/database/mirrored-database/azure-sql-database?wt.mc_id=studentamb_657938
connector overview	https://learn.microsoft.com/fabric/data-factory/connector-overview?wt.mc_id=studentamb_657938
Dataflows Gen2	https://learn.microsoft.com/fabric/data-factory/dataflows-gen2-overview?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
Fast Copy	https://learn.microsoft.com/fabric/data-factory/dataflows-gen2-fast-copy?wt.mc_id=studentamb_657938
Lakehouse tutorial	https://learn.microsoft.com/fabric/data-engineering/tutorial-lakehouse-introduction?wt.mc_id=studentamb_657938
ingest data into a warehouse	https://learn.microsoft.com/fabric/data-warehouse/ingest-data?wt.mc_id=studentamb_657938
Use Apache Spark in Fabric	https://learn.microsoft.com/training/modules/use-apache-spark-work-files-lakehouse/?wt.mc_id=studentamb_657938
Ingest with Dataflows Gen2	https://learn.microsoft.com/training/modules/use-dataflow-gen-2-fabric/?wt.mc_id=studentamb_657938
Load data into a warehouse	https://learn.microsoft.com/training/modules/load-data-into-microsoft-fabric-data-warehouse/?wt.mc_id=studentamb_657938
Real-Time hub	https://learn.microsoft.com/fabric/real-time-hub/real-time-hub-overview?wt.mc_id=studentamb_657938
Eventstream	https://learn.microsoft.com/fabric/real-time-intelligence/event-streams/overview?wt.mc_id=studentamb_657938
Eventhouse	https://learn.microsoft.com/fabric/real-time-intelligence/eventhouse?wt.mc_id=studentamb_657938
create a database	https://learn.microsoft.com/fabric/real-time-intelligence/create-database?wt.mc_id=studentamb_657938
add an Event Hubs source	https://learn.microsoft.com/fabric/real-time-intelligence/event-streams/add-source-azure-event-hubs?wt.mc_id=studentamb_657938
route events based on content	https://learn.microsoft.com/fabric/real-time-intelligence/event-streams/route-events-based-on-content?wt.mc_id=studentamb_657938
lakehouse	https://learn.microsoft.com/fabric/real-time-intelligence/event-streams/add-destination-lakehouse?wt.mc_id=studentamb_657938
Query acceleration	https://learn.microsoft.com/fabric/real-time-intelligence/query-acceleration?wt.mc_id=studentamb_657938
materialized view	https://learn.microsoft.com/kusto/query/materialized-view-function?view=microsoft-fabric&wt.mc_id=studentamb_657938
materialized views in Fabric	https://learn.microsoft.com/fabric/real-time-intelligence/materialized-view?wt.mc_id=studentamb_657938
table update policy	https://learn.microsoft.com/fabric/real-time-intelligence/table-update-policy?wt.mc_id=studentamb_657938
windowing functions	https://learn.microsoft.com/azure/stream-analytics/stream-analytics-window-functions?wt.mc_id=studentamb_657938
Real-Time Intelligence overview	https://learn.microsoft.com/fabric/real-time-intelligence/overview?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
KQL overview	https://learn.microsoft.com/kusto/query/?view=microsoft-fabric&wt.mc_id=studentamb_657938
summarize	https://learn.microsoft.com/kusto/query/summarize-operator?view=microsoft-fabric&wt.mc_id=studentamb_657938
bin()	https://learn.microsoft.com/kusto/query/bin-function?view=microsoft-fabric&wt.mc_id=studentamb_657938
Use Eventstream	https://learn.microsoft.com/training/modules/explore-event-streams-microsoft-fabric/?wt.mc_id=studentamb_657938
Work with real-time data in an Eventhouse	https://learn.microsoft.com/training/modules/query-data-kql-database-microsoft-fabric/?wt.mc_id=studentamb_657938
Use Activator	https://learn.microsoft.com/training/modules/use-fabric-activator/?wt.mc_id=studentamb_657938
Monitoring hub	https://learn.microsoft.com/fabric/admin/monitoring-hub?wt.mc_id=studentamb_657938
monitor pipeline runs	https://learn.microsoft.com/fabric/data-factory/monitor-pipeline-runs?wt.mc_id=studentamb_657938
monitoring view	https://learn.microsoft.com/fabric/real-time-intelligence/event-streams/monitor?wt.mc_id=studentamb_657938
Spark monitoring	https://learn.microsoft.com/fabric/data-engineering/spark-monitoring-overview?wt.mc_id=studentamb_657938
detail monitoring	https://learn.microsoft.com/fabric/data-engineering/spark-detail-monitoring?wt.mc_id=studentamb_657938
Query insights	https://learn.microsoft.com/fabric/data-warehouse/query-insights?wt.mc_id=studentamb_657938
Spark job definition	https://learn.microsoft.com/fabric/data-engineering/spark-job-definition?wt.mc_id=studentamb_657938
Activator rule	https://learn.microsoft.com/fabric/real-time-intelligence/data-activator/activator-create-activators?wt.mc_id=studentamb_657938
monitor a warehouse	https://learn.microsoft.com/training/modules/monitor-fabric-data-warehouse/?wt.mc_id=studentamb_657938
T-SQL surface area	https://learn.microsoft.com/fabric/data-warehouse/tsql-surface-area?wt.mc_id=studentamb_657938
troubleshoot Fabric Data Warehouse	https://learn.microsoft.com/fabric/data-warehouse/troubleshoot-fabric-data-warehouse?wt.mc_id=studentamb_657938
table maintenance	https://learn.microsoft.com/fabric/data-engineering/lakehouse-table-maintenance?wt.mc_id=studentamb_657938
Intelligent cache	https://learn.microsoft.com/fabric/data-engineering/intelligent-cache?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
Autotune	https://learn.microsoft.com/fabric/data-engineering/autotune?wt.mc_id=studentamb_657938
Statistics	https://learn.microsoft.com/fabric/data-warehouse/statistics?wt.mc_id=studentamb_657938
Result-set caching	https://learn.microsoft.com/fabric/data-warehouse/caching?wt.mc_id=studentamb_657938
Table clone	https://learn.microsoft.com/fabric/data-warehouse/clone-table?wt.mc_id=studentamb_657938
Time travel	https://learn.microsoft.com/fabric/data-warehouse/time-travel?wt.mc_id=studentamb_657938
warehouse performance guidelines	https://learn.microsoft.com/fabric/data-warehouse/guidelines-warehouse-performance?wt.mc_id=studentamb_657938
workload management	https://learn.microsoft.com/fabric/data-warehouse/workload-management?wt.mc_id=studentamb_657938
Introduction to end-to-end analytics using Fabric	https://learn.microsoft.com/training/modules/introduction-end-analytics-use-microsoft-fabric/?wt.mc_id=studentamb_657938
Get started with lakehouses	https://learn.microsoft.com/training/modules/get-started-lakehouses/?wt.mc_id=studentamb_657938
Work with Delta Lake tables	https://learn.microsoft.com/training/modules/work-delta-lake-tables-fabric/?wt.mc_id=studentamb_657938
Organize a lakehouse using medallion architecture	https://learn.microsoft.com/training/modules/describe-medallion-architecture/?wt.mc_id=studentamb_657938
Get started with data warehouses	https://learn.microsoft.com/training/modules/get-started-data-warehouse/?wt.mc_id=studentamb_657938
Get started with Real-Time Intelligence	https://learn.microsoft.com/training/modules/get-started-kusto-fabric/?wt.mc_id=studentamb_657938
Create Real-Time Dashboards	https://learn.microsoft.com/training/modules/create-real-time-dashboards-microsoft-fabric/?wt.mc_id=studentamb_657938
Data Factory	https://learn.microsoft.com/fabric/data-factory/tutorial-end-to-end-introduction?wt.mc_id=studentamb_657938
Real-Time Intelligence	https://learn.microsoft.com/fabric/real-time-intelligence/tutorial-introduction?wt.mc_id=studentamb_657938
warehouse	https://learn.microsoft.com/fabric/data-warehouse/tutorial-introduction?wt.mc_id=studentamb_657938
DP-600	https://learn.microsoft.com/credentials/certifications/fabric-analytics-engineer-associate/?wt.mc_id=studentamb_657938
what's new in Fabric	https://learn.microsoft.com/fabric/fundamentals/whats-new?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
Microsoft Fabric blog	https://www.microsoft.com/microsoft-fabric/blog/?wt.mc_id=studentamb_657938
Fabric Community	https://community.fabric.microsoft.com/?wt.mc_id=studentamb_657938
Microsoft Q&A	https://learn.microsoft.com/answers/products/?wt.mc_id=studentamb_657938